

Hermes Agent: An Architectural Study and Agentic Patterns Catalog

A senior-architect study of NousResearch/hermes-agent — code-verified against commit 4c9628e (snapshot July 2026)

Executive Summary

hermes-agent is Nous Research’s open-source AI agent: Python, MIT-licensed, published at github.com/NousResearch/hermes-agent. This study inspects exactly one snapshot — a shallow clone at HEAD 4c9628e, July 2026 — and every code claim is pinned to that commit by `path:line` and `permalink`. The README, website, and developer documentation are treated as claims to verify, not facts; where they conflict with the code, the code wins and the conflict is labeled.

The report’s thesis is that hermes-agent is a single synchronous ReAct-style loop whose complexity lives in self-authored data rather than orchestration structure. Against the orchestration-graph archetype, where behavior is topology fixed at authoring time, hermes-agent is a fixed interpreter over mutable content: skills hold procedures, memory files hold facts, cron jobs hold schedules, toolsets hold capabilities. There is no planner–executor split, no DAG engine, no tree search, no multi-agent debate — each absence verified in code. The bet buys compounding behavior with no code change; it costs controllability and observability, repaid with hard capability guards at the tool layer rather than more orchestration.

Three findings are what a skimming architect must retain. First, prompt-cache economics is the hidden architect: five otherwise unrelated subsystems — byte-identical history replay, date-only timestamps, the frozen memory snapshot, the learning fork’s cache-parity pins, idle-only async delivery — exist to keep the provider prefix byte-stable, and a clone targeting a provider without prefix caching can delete roughly 15–20% of the system’s complexity. Second, self-improvement is disciplined text gardening: the learning loop never mutates code, weights, or its own prompts; it writes bounded, human-readable Markdown through whitelisted tools, with fitness-measured optimization in a separate repository behind human-reviewed PRs. Third, capability subtraction beats instruction: a tool absent from the schema is physically uncallable, and every runaway or recursion risk is mitigated by removal in code, prompt-level hints a second tier only.

Five stale-source corrections, all resolved to code at HEAD: the loop is `run_conversation()` in `agent/conversation_loop.py`, not the `AIAgent._run_agent_loop()` that CONTRIBUTING.md still describes (C1); the Atropos/GRPO RL environments were removed on 2026-05-15 and survive only as seams (C2); `session_search` makes zero LLM calls despite the README’s “LLM summarization” claim (C3); a literal call-site count finds 74 statically registered tools against the README’s “40+” (C4); memory files live at `$HERMES_HOME/memories/`, not the config root older articles cite (C5).

The payoff is two deliverables. Chapter 10’s patterns catalog abstracts the mechanisms into 18 handbook-liftable entries, four graded PIONEER — agent-curated file memory, self-authored SKILL.md procedural memory, natural-language cron with isolated sessions, and capability-

subtraction guardrails. Its companion is a staged 10-step clone roadmap whose independent per-chapter estimates converge on one dependency order and one conclusion: the agent pattern is roughly 10–15% of the codebase; the rest is platform, providers, and product surface.

How to Read This Study

All file:line references and permalinks pin the July-2026 snapshot at HEAD 4c9628e, formatted `https://github.com/NousResearch/hermes-agent/blob/4c9628e/<path>#L<x>` — never bare `/blob/main/`, because main drifts and the documentation lags the code in three verified places (C1–C3). One claim cites live history rather than the snapshot: the RL-environment removal, verified through the GitHub commits API — removal commit 5af672c753, PR #26106, dated 2026-05-15. Three evidence grades are used throughout: **verified-in-code** (read in the snapshot, cited by file and line), **documented-but-unverified** (asserted by docs or third parties, unconfirmed in code), and **[INFERRED]**, preserved verbatim wherever the underlying research used it. Single-source counts — the 74 tools, 33 providers, curator cadences, and RPC limits — each carry their counting method so the reader can reproduce or dispute them. Bracketed `\[N^N\]` indices cite external literature and third-party sources, resolved against the numbered web-reference list appended at assembly. Line numbers rot faster than mechanisms; the mechanisms are the durable content, and the self-improvement and provider surfaces are flagged in-text as the most volatile. Two reading paths: pattern evaluators start with Chapter 10’s catalog, then Chapter 7’s learning loop; clone builders read Chapters 2–10 in order, since the roadmap cites that evidence base rather than re-deriving it.

1. Positioning and the Loop-Centric Thesis

1.1 The Subject and the Evidence Base

1.1.1 *hermes-agent: subject, license, snapshot*

The subject is hermes-agent, an open-source AI agent maintained by Nous Research: Python, MIT-licensed, published at `github.com/NousResearch/hermes-agent`. Every architectural claim is pinned to one snapshot — a shallow clone at HEAD ~4c9628e, July 2026 — and every code citation carries a path:line reference with permalinks pinned to `/blob/4c9628e/` rather than a moving branch.

The method matters more than the subject. The README, website, and developer documentation are treated as *claims to verify*, not facts. Each statement carries one of three evidence grades: **verified-in-code** (read in the snapshot, cited by file and line), **documented-but-unverified** (asserted by documentation or third-party sources, unconfirmed in code), and **[INFERRED]** (a conclusion drawn from code state; the marker is preserved verbatim wherever the underlying research used it). Single-source counts — the 74 registered tools are the canonical example — carry their counting method (“literal `registry.register()` call-site count”) so the reader can reproduce or dispute them.

1.1.2 Currency caveats

This repository moves fast, and its documentation lags its code in three verified places, each labeled where it first matters: the core loop's name and location (C1, Chapter 3), the removal of the Atropos/GRPO reinforcement-learning (RL) environments on 2026-05-15 via PR #26106 (C2, Chapter 9), and the removal of LLM summarization from session search (C3, Chapter 6). The most volatile sections — and the claims most likely to rot first — are the self-improvement mechanics (Chapters 6–7) and the channel integrations, which this report excludes partly because they churn. Line numbers rot faster than mechanisms; the mechanisms are the durable content.

1.2 The Central Architectural Bet

1.2.1 One loop, not a graph

The thesis of this report is that hermes-agent is a single, synchronous, ReAct-style loop whose complexity lives in self-authored data rather than in orchestration structure. The absence of alternatives is deliberate and verifiable: there is no planner–executor split, no directed-acyclic-graph (DAG) workflow engine, no tree search or backtracking over reasoning paths, no multi-agent debate anywhere in the tree. Where a graph framework would put a planner, hermes-agent puts loop-implicit planning plus kanban tools that externalize a plan as editable data; where tree search would branch reasoning, it offers checkpoint/rollback — environment-level undo instead of reasoning-level branching. This matches the field's convergence on the boring, debuggable, interruptible ReAct while-loop as the spine of serious agent harnesses.¹ The ABSENT section of Chapter 10, Part A treats each omission as a design decision, not a gap.

1.2.2 Behavior as data the loop reads

The forces are best stated as a trade. An orchestration-graph agent (LangGraph-style) encodes behavior as topology: nodes, edges, and state transitions that are inspectable and controllable but fixed at authoring time. Hermes Agent encodes behavior as *data the one loop reads*: skills hold procedures, memory files hold facts, cron jobs hold schedules, toolsets hold capabilities. The loop is a fixed interpreter; the behavior is mutable content. What this buys is emergent, compounding behavior — the agent rewrites its own procedures between sessions with no code change, and Chapters 6–7 show it doing so. What it costs is controllability and observability: there is no graph to visualize, no edge to breakpoint, and workflow correctness is prompt-mediated rather than structurally enforced. The system's answer is not more orchestration but hard capability guards at the tool layer (Chapters 4 and 8) and bounded, human-auditable mutation surfaces for everything the agent writes about itself (Chapter 7).

1.2.3 One AI Agent, several fronts — and the clone boundary

A consequence of the bet is that one platform-agnostic AI Agent class serves every entry point: the interactive CLI, the messaging gateway, batch runs, and the Agent Client Protocol (ACP)/API surface. Platform differences live in the entry points, not in the agent — which is why a clone can be scoped to the agent core with the channels named only to be excluded. Telegram, Discord, Slack, and the other adapters are delivery plumbing behind the gateway;

¹ KINNEY S. Agent loops. <https://github.com/stevekinney/stevekinney.net/blob/main/writing/agent-loops.md>

they contain no agentic patterns, and the clone roadmap in Chapter 10, Part B never touches them.

1.3 Component Decomposition Map

Figure 1 decomposes the snapshot into its load-bearing areas; the paragraphs below state each area’s responsibility at module-boundary level.

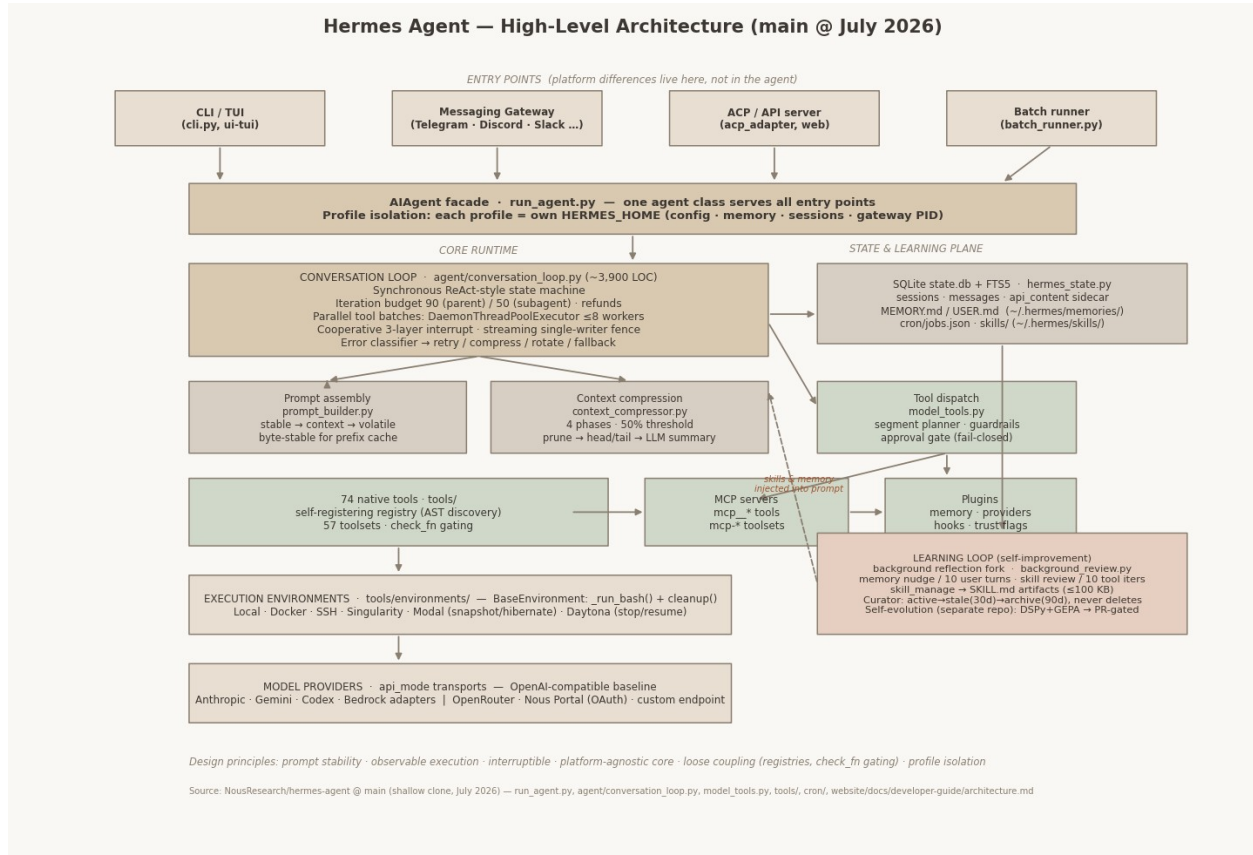


Figure 1: High-level architecture of hermes-agent at snapshot 4c9628e (July 2026) — entry points over a single AI Agent facade, the synchronous conversation loop at the core, and the state/learning plane beside it

1.3.1 Responsibilities per top-level area

agent/ is the runtime: the conversation loop itself (conversation_loop.py), prompt assembly and prefix-cache engineering (prompt_builder.py, system_prompt.py, prompt_caching.py), the background reflection fork (background_review.py), the Curator (curator.py), and the provider glue — transport adapters, the auxiliary client lane, the centralized error classifier. **tools/** is the capability plane: a self-registering registry, 35 tool modules carrying registration call sites (per the literal call-site count in Chapter 4), and tools/environments/, where six terminal backends sit behind a two-method BaseEnvironment contract. **cron/** owns proactivity: natural-language schedule parsing, the job store, the scheduler tick, the isolated per-fire session executor. **hermes_state.py** is the persistence plane: one SQLite state.db per profile in write-ahead-logging (WAL) mode, with FTS5 (SQLite full-text search) indexes over

session history. **hermes_cli/** is the operator surface: the CLI plus `auth.py`, which holds the provider-resolution priority chain. Everything above the AI Agent facade in the figure is an entry point; everything below the tool layer is a backend the loop does not see.

1.3.2 The god-file decomposition, and a stale-document warning

The shape of the runtime today is the residue of a refactor, and the module header at `agent/conversation_loop.py:1-8` ([permalink](#)) says so in its own words:

```
"""The agent conversation loop – extracted from ``run_agent.AIAgent``.
```

```
This is the biggest single chunk pulled out of ``run_agent.py``: the
roughly 3,900-line :func:`run_conversation` body that drives one user
turn through the agent (model call, tool dispatch, retries, fallbacks,
compression, post-turn hooks, background memory/skill review nudges).
```

```
The function takes the parent ``AIAgent`` instance as its first
```

The extraction was mechanical rather than architectural: `AIAgent.__init__` at `run_agent.py:498-500` ([permalink](#)) is a one-line forwarder — `"""Forwarder – see ``agent.agent_init.init_agent``. """` — and dozens of methods on the class are similar pass-throughs. The loop's mass moved; it was not reduced.

Stale-source note (C1). `CONTRIBUTING.md` still names the core loop

`AIAgent._run_agent_loop()`, but the loop actually lives at `agent/conversation_loop.py:669-6166`. Chapter 3 resolves the discrepancy (§3.1).

1.4 Architectural Positioning vs Peer Agents

1.4.1 Comparison at the architecture level

The table positions hermes-agent against three reference points at the pattern level only — loop, tools, memory, self-improvement, proactivity. It is deliberately not a feature comparison, and it carries no benchmark claims.

Dimension	hermes-agent	OpenClaw	Claude Code	AutoGPT
Loop topology	Single synchronous ReAct loop; behavior as data	Gateway daemon routing to agents; serialized per-session queues	ReAct queryLoop(), ephemeral per session	Continuous autonomous loop
Tool model	Self-registering import-time	Skills plus plugin tools	Hardwired built-ins plus	Hardwired command

Dimension	hermes-agent	OpenClaw	Claude Code	AutoGPT
	registry; schema- emission gating	behind the gateway	MCP	registry
Memory model	Agent-curated files + FTS5 session store	Workspace Markdown; optional hybrid vector search	CLAUDE.md hierarchy; subagent memory scopes	Vector store
Self- improvement	Background reflection fork + Curator; external PR- gated evolution	None equivalent	Absent	Absent
Proactivity	Natural- language cron with isolated sessions	Heartbeat + cron	None (session- scoped)	Continuous mode

Three observations give the table its meaning. First, the differentiating row is *self-improvement*: OpenClaw, a gateway-first Node.js daemon whose center of gravity is channel routing across twenty-plus messaging surfaces,^{2 3} has no equivalent of hermes-agent’s background reflection fork, Curator, or external evolution loop; Claude Code, an ephemeral-per-session coding harness with deny-first permissioning, likewise has no persistent learning mechanism.^{2 4} Second, convergence is real in the boring rows: every mature system here runs some ReAct-flavored loop,¹ and Claude Code’s subagents now carry per-agent memory scopes resembling hermes-agent’s curated-file pattern.⁵ Third, AutoGPT is the cautionary ancestor: it popularized the continuous autonomous loop with its canonical failures — infinite loops, unrecognized repeated actions, runaway cost⁶ — and hermes-agent reads as the same bet re-made with iteration budgets, approval tiers, and fail-closed scheduling. Where OpenClaw pairs heartbeat with cron,⁷ hermes-agent commits to cron alone, each fire an isolated session.

1.4.2 Shared vocabulary: the CoALA memory frame

The memory rows above anticipate a vocabulary the report uses consistently: the CoALA framework’s four memory types — **working** (the context window), **episodic** (what happened), **semantic** (what is true), **procedural** (how to act).^{8 9} Hermes Agent has a distinct

² Dive into Claude Code: The Design Space of Today’s and Future AI Agent Systems. <https://arxiv.org/html/2604.14228v2>

³ freeCodeCamp. How to Build and Secure a Personal AI Agent with OpenClaw. <https://www.freecodecamp.org/news/how-to-build-and-secure-a-personal-ai-agent-with-openclaw/>

⁴ CHEN G. Claude Code’s Five-Layer Architecture. <https://chenguangliang.com/en/posts/claude-code-five-layer-architecture/>

⁵ Subagent persistent memory scopes (user/project/local); MoA/voting parallelization per Anthropic taxonomy.

<https://chenguangliang.com/en/posts/claude-code-five-layer-architecture/>

⁶ vectara. AutoGPT planning failures case study (awesome-agent-failures). <https://github.com/vectara/awesome-agent-failures/blob/main/docs/case-studies/autogpt-planning-failures.md>

⁷ Clawlify. Heartbeat vs Cron in OpenClaw. <https://www.clawlify.com/resources/heartbeat-vs-cron-openclaw-guide-2026>

⁸ Octamem. The 4 Types of AI Agent Memory: Semantic, Episodic, Procedural & Working. <https://octamem.com/blog/types-of-ai-agent-memory>

⁹ mer.vin. AI Agent Memory Explained: Episodic, Semantic, Procedural and Working Context. <https://mer.vin/2026/05/ai-agent-memory-explained-episodic-semantic-procedural-and-working-context/>

mechanism for each: tiered prompt assembly and compression for working memory (Chapter 5), the SQLite/FTS5 session store for episodic memory and agent-curated MEMORY.md/USER.md files for semantic memory (both Chapter 6), and self-authored SKILL.md skills for procedural memory (Chapter 6). Later chapters argue in these terms rather than re-deriving definitions.

1.4.3 Reader's map

Two reading paths serve two audiences. **Profile A — the pattern evaluator:** read Chapter 10, Part A (the patterns catalog) first, then Chapter 7 (the learning loop, the deepest single system), then return to this chapter's thesis as the frame that makes the catalog cohere. **Profile B — the clone builder:** read Chapters 2–10 in order — loop, tools, context economics, memory, skills, learning, delegation, security and providers, research seams — because Chapter 10, Part B's staged build roadmap cites that evidence base rather than re-deriving it.

Either path passes next through Chapter 2, the study's interpretive layer: seven cross-cutting insights stated there without argument and argued in full in the host chapters each insight names.

2. Cross-Cutting Insights

2.1 The Seven Insights

This chapter is the study's interpretive layer: it states and does not argue. Each insight derives from at least two independent dimensions of the analysis and says so in one clause; each reappears as a callout in the host chapter(s) named in its parenthetical, where the code evidence is argued in full.

2.1.1 Prompt-cache economics is the hidden architect

Five otherwise unrelated subsystems — the SQLite `api_content` sidecar replaying history byte-identically ([agent/conversation_loop.py:1022](#)), three-tier prompt assembly with date-only timestamps, the frozen once-per-session memory snapshot, the learning fork's cache-parity pins ([agent/background_review.py:765](#)), and idle-only delivery of `async` completions — each exist to keep the provider prompt prefix byte-stable so KV caches hit. Derived from the storage, prompt, memory, learning, and delegation dimensions jointly, this is a load-bearing constraint, not an optimization: it explains choices a cloner would otherwise find bizarre, such as refusing fresh memory mid-session. Symmetrically, a clone targeting a provider without prefix caching can delete roughly 15–20% of the system's complexity. (Hosts: ch5, ch6, ch7, ch8, ch9.)

2.1.2 Loop, not graph

Hermes keeps one synchronous ReAct-style loop and encodes behavior as data the loop reads — skills as procedures, memory as facts, cron jobs as schedules, toolsets as capabilities — where graph-based frameworks encode behavior as topology. Derived from the loop dimension and the taxonomy dimension's verified absences (no planner-executor split, no DAG workflows, no tree search, no debate), this is the report's central architectural bet: agent as loop plus self-authored data. Every later chapter is its evidence, so it is named here and not argued. (Stated in ch1; evidence throughout.)

2.1.3 Self-improvement is disciplined text gardening

The learning loop never mutates code, weights, or its own prompts at runtime; it writes bounded, human-readable Markdown through whitelisted tools. Derived from the skills, memory, and learning dimensions, the transferable lesson is the guardrail stack — tool whitelist, provenance ContextVar, size caps, read-before-write, archive-never-delete, snapshot/rollback — which is cheap to copy; the only fitness-measured optimization lives in a separate repository and lands via human-reviewed PRs. Safety is achieved by constriction of the mutation surface, not by policing a general one; with no in-loop verifier beyond verify-on-stop, quality degrades gracefully with model quality. (Host: ch7.)

2.1.4 Capability subtraction beats instruction

Every recursion or runaway risk is mitigated by removing capability in code — a tool absent from the schema is a tool the model physically cannot call — rather than by instructing the model: the subagent blocklist ([tools/delegate_tool.py:46](#)), the cron toolset strip ([cron/scheduler.py:156](#)), and the frozen-at-import privilege toggles are the three canonical instances. Derived from the delegation, cron, and platform dimensions, the rule is: hard guards at the toolset layer, soft guards at the prompt layer — and Hermes chooses hard, prompt-level hints serving only as a second tier. A check_fn-style gate is roughly fifty lines and eliminates the entire runaway-automation class. (Hosts: ch4, ch8, ch9.)

2.1.5 Concentric extensibility rings

Hermes’s extension surfaces form concentric rings — native registry, toolsets, provider plugins, MCP servers, skills — across which capability, cost, and trust decrease together moving outward: trusted code, code with fail-closed trust flags, external processes, scanned untrusted text, and finally agent-authored text with provenance. Derived from the tool, provider, and skills dimensions, the decisive property is that the agent’s self-modification is confined to the outermost, cheapest, safest ring — the only ring the learning loop requires. (Hosts: ch4, ch6, ch9.)

2.1.6 Research-readiness as seams, proven by amputation

The Atropos/GRPO reinforcement-learning layer was removed in a single pull request (#26106, May 2026) without destabilizing the runtime, because its coupling points were narrow, named seams — register_task_env_overrides ([tools/terminal_tool.py:1127](#)), per-task_id sandbox isolation, the _run_async bridge — rather than a woven subsystem. Derived from the research-infrastructure and loop dimensions, the lesson is to couple training and eval through injectable seams that can be added or removed without forking the runtime. The removal is verified against live commit history; its motive and impact remain [INFERRED]. (Host: ch9.)

2.1.7 The minimal viable clone is 10–15% of the tree

Every dimension’s clone guidance independently converges on the same small, additive core in the same dependency order: provider path, loop, registry and tools, environment plus approval, context compression, memory, skills, learning fork, delegation, cron. Derived from all twelve dimensions’ independent estimates, this convergence — not any single line count — is the

evidence that roughly 10–15% of the codebase carries the agent pattern; the rest is platform, providers, and product surface. The staged build roadmap with per-stage estimates and deferrals is delivered as ch10 Part B. (Host: ch10 Part B.)

4. The Tool System

The loop-centric thesis of chapter 1 becomes concrete in the tool system: capabilities in hermes-agent are data — registry entries carrying schema, handler, toolset membership, and an availability probe — not branches in the loop’s control flow. Everything the model can do passes through one singleton ToolRegistry (tools/registry.py:765; [permalink](#)), and everything in this chapter is a consequence of that decision: how tools are discovered, how they are hidden, how they execute, and how the surface extends without touching core code.

4.1 Registration and Discovery

4.1.1 Self-registration at import time, AST-gated discovery

Each module in tools/ calls registry.register() at module top level, declaring a ToolEntry whose slots include name, toolset, schema, handler, check_fn, requires_env, is_async, max_result_size_chars, dynamic_schema_overrides (tools/registry.py:87–116). The dependency chain is deliberately acyclic: tools/registry.py imports nothing in-repo; tool modules import the registry; model_tools.py imports both and triggers discovery at its own import (model_tools.py:194–217; [permalink](#)). The force being traded is explicit: import-time registration pays startup cost and import side effects — every tool module’s top-level code runs whether or not the session enables it — in exchange for zero-config discoverability, with no import list to maintain.

Discovery is not a hardcoded list. discover_builtin_tools() scans tools/*.py and imports only modules that provably self-register, using a cheap text prefilter ahead of ast.parse. The gate, tools/registry.py:43–84 ([permalink](#)):

```
if "registry" not in source or "register" not in source:
    return False
tree = ast.parse(source, filename=str(module_path))
...
return any(_is_registry_register_call(stmt) for stmt in tree.body)
```

Three details carry the design intent. Only module-body statements count, so helpers that call register() inside a function are skipped by construction (tools/registry.py:44–48). mcp_tool.py is excluded; MCP registration has its own dynamic path (§4.2.2). Import failures are swallowed with a warning, so a missing optional dependency such as fal_client degrades one toolset rather than the process (tools/registry.py:78–83). The countervailing force: AST gating is a brittle heuristic — a module registering conditionally or through an alias would be silently skipped — accepted because it avoids heavy imports of modules that cannot register anything. MCP discovery was moved out of module import entirely because it can block for 120 s and the gateway lazy-imports model_tools inside its event loop; each entry point calls discover_mcp_tools() explicitly at startup (model_tools.py:199–210).

4.1.2 Stale-source note (C4): the real count

The README and website describe “40+ tools.” A literal `registry.register()` call-site count across 35 tools/*.py files at HEAD yields **74 statically registered built-in tools, 57 static toolsets** in `TOOLSETS`, of which **25 are hermes-*** platform bundles** — plus an unbounded number of dynamic MCP and plugin tools at runtime. The figure is Medium-confidence in the report’s grading: a single counter, but a deterministic method (cross-verification, conflict zone C4). “40+” is conservative marketing that remains literally true; this study’s count of record is 74, resolved to code at HEAD, snapshot July 2026.

4.2 Schemas, Toolsets, Gating

4.2.1 Schema emission and `check_fn` gating

Schemas are plain OpenAI function-calling dicts defined as module constants; the registry wraps each in the `{"type": "function", "function": ...}` envelope at emission time, after filtering and after merging any `dynamic_schema_overrides` — a zero-arg callable re-evaluated on every pass, so `delegate_task`’s description can reflect live concurrency limits (`tools/registry.py:558–576`; [permalink](#)). Between name resolution and wrapping sits the `check_fn` probe: an availability predicate behind a 30 s TTL cache with a 60 s “last-good” grace window, so a flaky Docker, Modal, or Playwright probe does not oscillate the schema (`tools/registry.py:143–206`). A representative call site, `tools/file_tools.py:2104–2107` ([permalink](#)):

```
registry.register(name="read_file", toolset="file",
schema=READ_FILE_SCHEMA, handler=_handle_read_file,
check_fn=_check_file_reqs, emoji=" ", max_result_size_chars=100_000)
registry.register(name="write_file", toolset="file",
schema=WRITE_FILE_SCHEMA, handler=_handle_write_file,
check_fn=_check_file_reqs, emoji=" ", max_result_size_chars=100_000)
registry.register(name="patch", toolset="file", schema=PATCH_SCHEMA,
handler=_handle_patch, check_fn=_check_file_reqs, emoji=" ",
max_result_size_chars=100_000)
registry.register(name="search_files", toolset="file",
schema=SEARCH_FILES_SCHEMA, handler=_handle_search_files,
check_fn=_check_file_reqs, emoji=" ", max_result_size_chars=100_000)
```

The four file tools share one probe, so the per-callable cache evaluates it once per schema build. The architectural weight sits here: a tool whose check fails is omitted from the emitted schema, and a tool absent from the schema is physically uncallable — the model cannot invoke what it cannot see. This is insight 4 (capability subtraction beats instruction) at its smallest scale: gating enforced in data at emission time, not by prompt-level “do not use” instructions a model can ignore.

4.2.2 Toolset composition, platform bundles, MCP merge, Tool Search

Toolsets are static dicts of shape `{description, tools, includes}`, with `includes` composing under cycle detection (`toolsets.py:689–769`); registry-only toolsets introduced by

plugins and MCP coexist with the static table. The 25 hermes-* bundles package capability per deployment: hermes-cli and hermes-cron are exactly the 54-name _HERMES_CORE_TOOLS (toolsets.py:31-81, 432-447), hermes-discord adds the discord pair, hermes-gateway composes the messaging bundles via includes, and hermes-webhook is intentionally minimal — four read-only tools — as prompt-injection hardening (toolsets.py:86-91). Disabling a platform bundle subtracts only its non-core delta so shared core tools survive (model_tools.py:416-438).

MCP servers merge into the same registry under toolset mcp-<server>, names prefixed mcp_{server}_{tool}, with per-server include/exclude filters and a prompt-injection scan of each description at registration (tools/mcp_tool.py:5462-5570). The collision policy is asymmetric by design — MCP may never shadow a built-in, while MCP→MCP overwrites are permitted for server refresh. The guard, tools/mcp_tool.py:5508-5515 ([permalink](#)):

```
existing_toolset = registry.get_toolset_for_tool(tool_name_prefixed)
if existing_toolset and not existing_toolset.startswith("mcp-"):
    logger.warning(
        "MCP server '%s': tool '%s' (→ '%s') collides with built-in "
        "tool in toolset '%s' – skipping to preserve built-in",
        name, mcp_tool.name, tool_name_prefixed, existing_toolset,
    )
    continue
```

On notifications/tools/list_changed a server deregisters and re-registers its whole surface — “nuke and repave” — and shutdown deregisters, so dead servers leave no phantom schemas (tools/mcp_tool.py:3452-3467). Tool Search then applies progressive disclosure to this volatile surface: when MCP/plugin tools exceed roughly 10 % of the context window, non-core tools collapse behind three synthesized bridge tools (tool_search, tool_describe, tool_call); core tools are never deferred (model_tools.py:551-579). The prompt-token economics of that threshold are chapter 5’s subject.

4.3 Execution Pipeline

4.3.1 Planner → executor → guardrails → approval

An assistant batch is first split by _plan_tool_batch_segments() into maximal contiguous parallel runs and sequential barriers, preserving emission order exactly (agent/tool_dispatch_helpers.py:40-205). The safety rules are declarative: clarify is always a barrier; eleven read-only tools are unconditionally parallel-safe; the path-scoped tools (read_file, write_file, patch) join a run only on non-overlapping canonical target paths; MCP tools join only if their server declared supports_parallel_tool_calls; runs shorter than two calls are demoted. Strategy selection falls out of the segmentation — single call: sequential; one parallel segment: concurrent; mixed: segmented — and execution lands on the custom DaemonThreadPoolExecutor introduced in chapter 3 (≤8 workers, 420 s batch deadline, abandoned rather than joined on timeout so wedged threads cannot stall CLI exit; agent/tool_executor.py:95-137, 836-845). All argument parsing and block decisions complete before any worker starts. Around the dispatch sit the guardrails — a per-turn circuit breaker issuing allow/warn/block/halt decisions on repeated failure signatures

(agent/tool_guardrails.py:63-173) — and the approval chain-of-responsibility, which lives mostly inside the terminal and execute_code handlers rather than in the generic dispatch path: container skip → hardline floor → sudo-stdin guard → user deny rules → yolo/allowlist → pattern and content scanning → prompt (tools/approval.py:3180-3235; full treatment in chapter 9). A plugin pre_tool_call hook can escalate any tool into the same human gate via request_tool_approval. One responsibility ambiguity is worth naming: dangerous-command policy is a property of the tool, not the pipeline, so any path bypassing handle_function_call — the execute_code RPC channel is the existing example — must re-implement its own guard, which it does via whole-script pre-execution approval.

4.3.2 Extension story and toolset taxonomy

Adding a built-in tool is a two-file change, verified against the developer guide and the code: create tools/your_tool.py containing a check_fn probe, a handler returning a JSON string with errors as {"error": ...} (never raised — the contract is enforced by _normalize_handler_result, tools/registry.py:583-612), an OpenAI-format schema dict, and a module-level registry.register(); then add the name to a toolset in toolsets.py. No discovery step exists — the AST gate picks the module up automatically (website/docs/developer-guide/adding-tools.md:29-211). The taxonomy below condenses the 74 static tools into the fourteen categories of the tools/ inventory.

Category	Representative tools	Count	Bundle membership
File & patch	read_file, write_file, patch, search_files	4	Core → all hermes-* bundles except hermes-webhook
Terminal & code execution	terminal, process, execute_code	3	Core, except hermes-webhook
Browser automation	browser_navigate ... browser_dialog	12	Core
Web, search, media & vision	web_search, web_extract, x_search, image_generate, video_generate	9	Mixed: four in core (three also in hermes-webhook); x_search/video opt-in
Audio & voice	text_to_speech	1	Core
Skills system	skills_list, skill_view, skill_manage	3	Core
Planning, memory & multi-agent	todo, memory, clarify, delegate_task, kanban_*	16	Core; kanban check_fn-gated on worker env
Messaging & platform	discord, feishu_drive_*, yb_*, cronjob, session_search	14	Mixed: discord pair only in hermes-discord; cronjo b/session_search

Category	Representative tools	Count	Bundle membership
Smart home	ha_get_state, ha_call_service	4	core Core-listed, default-off on cron, HASS_TOKEN-gated
Desktop GUI & computer use	read_terminal, project_*, computer_use	8	Mixed: pane tools core (HERMES_DESKTOP-gated); project_* GUI-gateway only
Registry & orchestration core	tool_search, tool_describe, tool_call (synthesized)	—	Infrastructure; emitted by Tool Search, not TOOLSETS
Approval & security	— (gates, not tools)	—	Cross-cutting guard layer
MCP client infrastructure	mcp__<server>__<tool> (runtime)	dynamic	Registry-only mcp-* toolsets with server-name aliases
Misc/support	—	—	Internal helpers

Two facts stand out. First, the surface is heavy at the edges: browser automation (12) and planning/multi-agent (16, of which kanban alone contributes 12) are the largest clusters, while the classic agent primitives — files, terminal, web — are comparatively small; the system’s ambition shows in where the tool mass sits. Second, bundle membership is mostly a function of one list: ten of the fourteen categories feed `_HERMES_CORE_TOOLS`, so platforms differentiate by subtraction (`hermes-webhook` drops to four tools) and by `check_fn` gating (home assistant, kanban, desktop, computer use are core-listed but hidden until their probe passes) rather than by maintaining divergent per-platform lists. The single-source caveat applies to the counts: they derive from the literal call-site count of §4.1.2, and the infrastructure rows carry no static registrations — the Tool Search bridge tools are synthesized at schema-emission time and MCP tools exist only at runtime.

4.3.3 Extensibility rings

The registration, gating, and merge mechanisms above compose into the report’s fifth cross-cutting insight, whose primary home is this chapter: extensibility in `hermes-agent` is concentric.

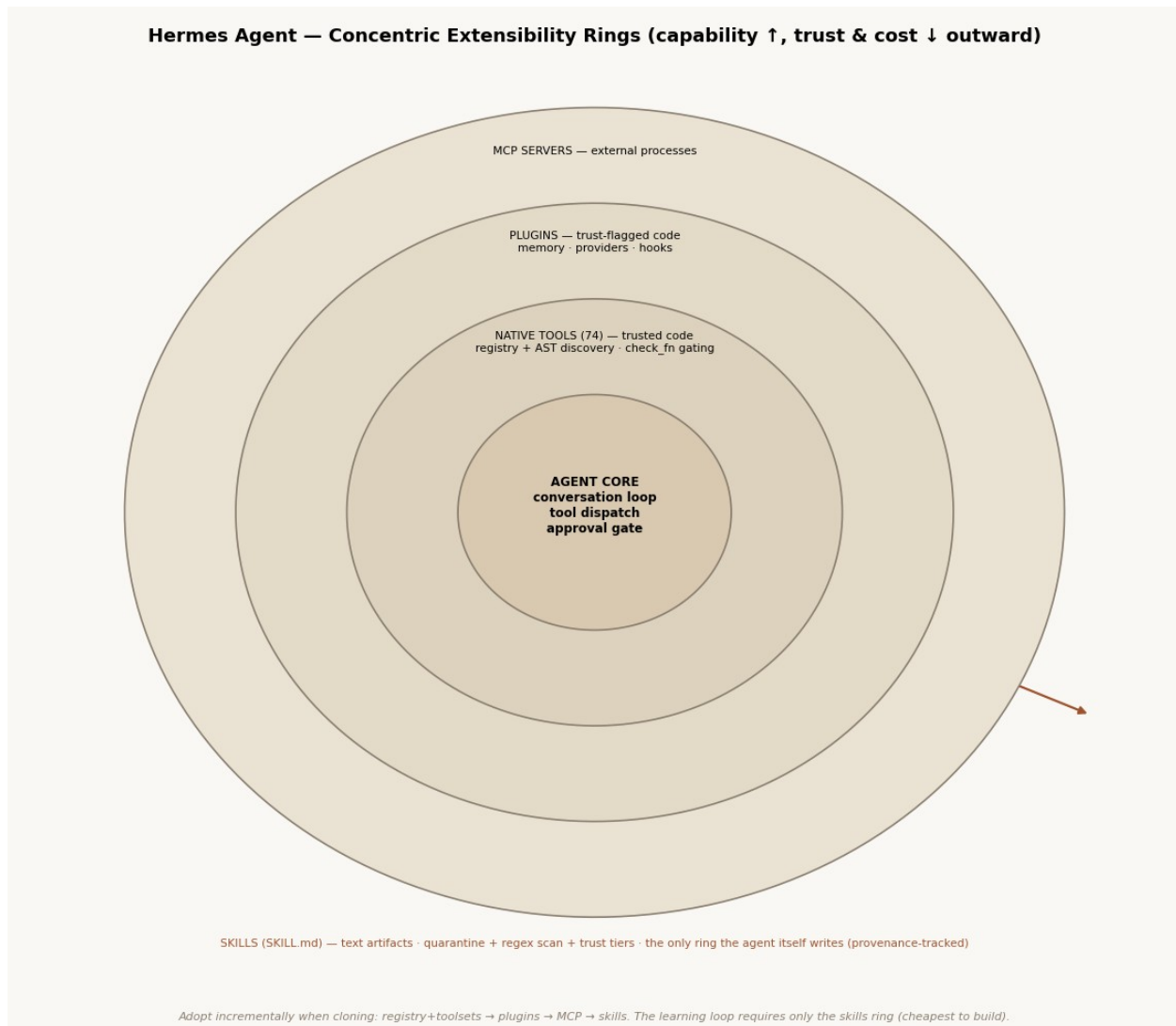


Figure 4: Concentric extensibility rings — registry → toolsets → plugins → MCP → skills; capability, integration cost, and trust all decrease moving outward, and the agent’s own writes are confined to the outermost ring.

Moving outward through Figure 4: **native tools** are trusted in-repo code admitted by the AST gate; **toolsets** package that code into per-platform capability surfaces; **plugins** are external code admitted with trust flags — overriding an existing tool requires explicit operator opt-in (`plugins.entries.<id>.allow_tool_override`), bound to the handler’s defining module, with a matching gate on `deregister()` to prevent bypass-by-delete (`tools/registry.py:316–347, 459–524`); **MCP servers** are external processes whose descriptions are scanned for injection, whose collisions are skipped, and whose parallelism is opt-in; **skills** are untrusted-but-scanned text, admitted through quarantine and trust tiers — and agent-authored skills, the only ring the agent itself writes, carry provenance tracking (chapter 6). Capability, cost, and trust decrease together moving outward. The design consequence: self-modification is confined to the cheapest, least trusted ring — the loop never gains a new native tool at runtime, but it can gain a new procedure. A clone can adopt the rings

incrementally (registry and toolsets first, skills last), yet the learning loop of chapter 7 requires only the skills ring.

4.3.4 Clone notes

Clone notes. *The registry plus a minimal toolset is milestone three of the build roadmap, immediately after the provider path and the core loop: roughly 600 lines for the registry and 1,500 for ten to fifteen core tools. A static import list is an acceptable v1 simplification; the AST gate buys discoverability only once third-party modules exist. Two pieces should not be simplified away. First, implement `check_fn`-style gating at schema-emission time early — about 50 lines eliminates the entire class of runaway-capability bugs that prompt-level instructions merely discourage. Second, keep the handler contract exact: JSON string out, errors as `{"error": ...}` and never raised, thread-safe if the tool can appear in parallel batches. The 64-line daemon pool (`tools/daemon_pool.py`) is worth copying verbatim — it exists because wedged tool threads otherwise produce multi-minute CLI exits.*

3. The Core Runtime: The Conversation Loop

Chapter 1 stated the thesis — Hermes Agent is a loop, not a graph — and chapter 2 named it as the report’s central architectural bet (insight 2): behavior is encoded as data the loop reads, not as orchestration topology. This chapter supplies the primary evidence: a dissection of the one function that executes every user turn, reproducible against the pinned snapshot (HEAD 4c9628e, July 2026).

3.1 Where the Loop Actually Lives

The core loop is not a method on the agent class. It is the module-level function `run_conversation(agent, ...)` spanning `agent/conversation_loop.py:669–6166` — roughly 3,900 lines in a 6,170-line module ([permalink](#)). The `AIAgent` class in `run_agent.py` reaches it through a thin forwarder, `AIAgent.run_conversation` (`run_agent.py:6613–6668`), which publishes two `ContextVars` — a conversation tag and session accounting handles — so that every auxiliary LLM call inside the turn inherits cost attribution, and then delegates. `AIAgent` itself is now largely a facade: its `__init__` forwards to `agent.agent_init.init_agent` (`run_agent.py:498–500`, [permalink](#)), and dozens of methods are one-line forwarders into `agent/*` modules. The loop module’s own docstring names the refactor: “the biggest single chunk pulled out of `run_agent.py`.”

Stale-source note (C1). *CONTRIBUTING.md still describes the architecture as “User message → `AIAgent.run_agent_loop()`” (`CONTRIBUTING.md:304`, [permalink](#)). No symbol named `_run_agent_loop` exists anywhere in the tree at this snapshot (verified by recursive `grep` over `*.py`). The `CONTRIBUTING` diagram predates the god-file decomposition that extracted the loop into `agent/conversation_loop.py`. Resolution: code at HEAD, snapshot July 2026, is authoritative; any documentation or third-party article that locates the loop in `run_agent.py` is temporally stale.*

This matters beyond citation hygiene. A reader cloning “the loop” from the documented location would extract a facade; the actual control flow, recovery logic, and budget enforcement all live in the extracted module.

3.2 Turn Lifecycle

3.2.1 Prologue seam, two nested loops, epilogue seam

A single user turn passes through four regions (Figure 2). First, the **prologue seam**: `build_turn_context(...)` (`agent/turn_context.py`, invoked at `conversation_loop.py:729–748`) performs — per its module docstring — stdio guarding, retry-counter resets, message sanitization, system-prompt restore-or-build, session-row creation, preflight compression, the `pre_llm_call` plugin hook, and crash-resilience persistence. It also binds the execution thread for interrupt targeting (`agent._execution_thread_id`, `turn_context.py:1069`). The system-prompt step reuses the byte-identical prompt stored in the session row when present — cache-stability machinery owned by chapter 5.

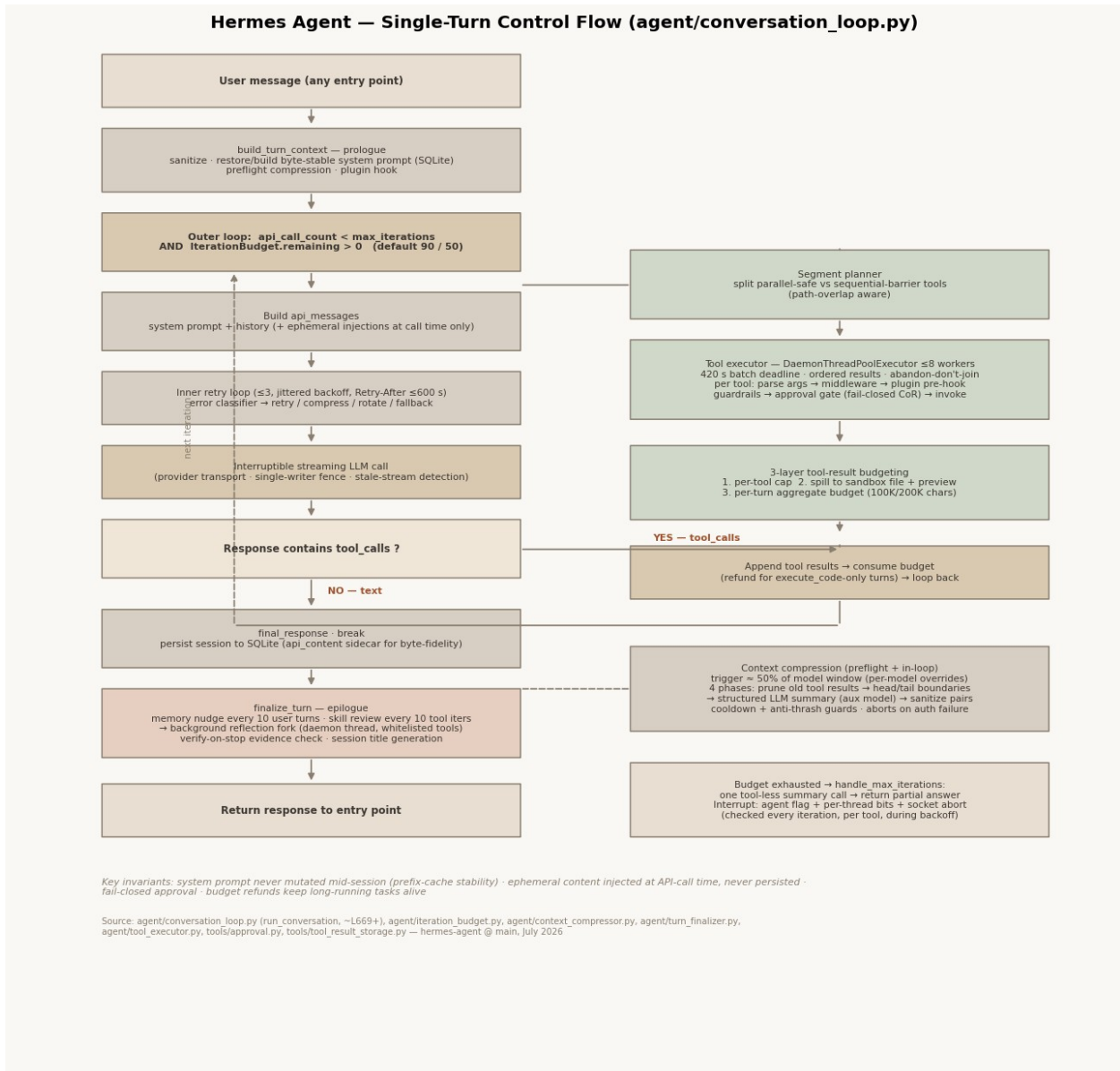


Figure 2: Turn control flow — prologue seam, outer budget loop, inner retry loop, epilogue seam

Second, the **outer budget loop** (conversation_loop.py:822), where one iteration equals one model API call. Its skeleton at conversation_loop.py:822–857 shows the two gates every iteration must pass:

```
while (api_call_count < agent.max_iterations and
agent.iteration_budget.remaining > 0) or agent._budget_grace_call:
    ...
    if agent._interrupt_requested:
        _turn_exit_reason = "interrupted_by_user"
        break
    ...
    elif not agent.iteration_budget.consume():
```

Each iteration rebuilds `api_messages` (system-prompt prepend, MoA advisory injection, Anthropic cache_control markers), runs preflight context-pressure checks, then enters the **inner retry loop** (while `retry_count < max_retries`, `conversation_loop.py:1453`; default 3, config `agent.api_max_retries`). The API call passes through a middleware seam (`run_llm_execution_middleware`, `conversation_loop.py:1687–1714`) into an interruptible wrapper. A response carrying `tool_calls` is dispatched through `agent._execute_tool_calls(...)` (`conversation_loop.py:5394` → `run_agent.py:6449–6491`), whose segment planner splits the batch into parallel-safe runs separated by sequential barriers — the dispatch pipeline is chapter 4’s subject; what matters here is that after tools execute, the loop continues, and only a tool-call-free response ends the iteration with a `final_response`.

Fourth, the **epilogue seam**: every exit path funnels into `finalize_turn(...)` (`conversation_loop.py:6146–6166` → `agent/turn_finalizer.py:69`), which handles the budget-exhaustion fallback (§3.3.1), trajectory save, session persistence, and transcript-tail invariants. The two seams are the system’s primary extension points: both were extracted behavior-neutrally (their docstrings say so), so a cloner can replace prologue or epilogue policy without touching the loop body.

3.2.2 The implicit state machine

There is no state enum. Control state is carried by two artifacts: the diagnostic string `_turn_exit_reason`, initialized to "unknown" at `conversation_loop.py:784` and overwritten at every exit path, and `TurnRetryState` (`agent/turn_retry_state.py:32–79`), a per-iteration dataclass collapsing roughly sixteen one-shot recovery guards plus four `restart_with_*` signals (compressed messages, length continuation, rebuilt messages, redirected messages) that the loop reads after each attempt to decide whether to rebuild the request and re-enter. Transitions are `break/continue` plus these flags; the comments frame the flags as “restart signals ... read by the loop after the attempt,” which reads as deliberate design [INFERRED: deliberate — no explicit state enum exists].

The exit reasons observed at this snapshot:

Exit reason	Set at	Meaning	Restart behavior
<code>text_response(finish_reason=...)</code>	<code>conversation_loop.py:6046</code>	Normal completion; model answered without tool calls	None; loop breaks, finalizer assembles result
<code>budget_exhausted</code>	<code>conversation_loop.py:854</code>	<code>IterationBudget.consume()</code> returned <code>False</code> at loop top	No in-loop restart; finalizer may issue one tool-less summary call
<code>max_iterations_reached(n/max)</code>	<code>turn_finalizer.py:124–131</code>	Finalizer’s relabeling of budget exits after the summary fallback	None; result carries the summary or a preserved verification answer
<code>interrupted_by_user</code>	<code>conversation_loop.py:839</code>	Interrupt flag seen between iterations	Cooperative break; budget fallback explicitly ineligible

Exit reason	Set at	Meaning	Restart behavior
interrupted_during_a_pi_call	conversation_loop.py:4708	Interrupt landed mid-request; dangling tool_calls patched closed	Cooperative break after transcript repair
guardrail_halt	conversation_loop.py:5398	Tool-guardrail / repeated-invalid-call circuit breaker tripped	Hard break; no retry
all_retries_exhausted_no_response	conversation_loop.py:4763	Inner retry loop spent without a usable response	Break; failure result dict
partial_stream_recovery / fallback_prior_turn_content	conversation_loop.py:5547, 5578	Empty final response recovered from partial stream or prior turn	Recovery supplies final_response; loop exits normally
empty_response_exhausted	conversation_loop.py:5760	Empty-response nudges hit their attempt cap	Break with failure semantics
ollama_runtime_context_too_small / runtime-context error	conversation_loop.py:1240	Local-runtime context floor violation	Iteration refunded (L1243–1246) before exit
local_processing_error(...) / error_near_max_iterations(...)	conversation_loop.py:6136–6139	Outer safety-net classifier judged the exception a deterministic local bug	Deliberately not retried — “they will fail identically on every iteration and only burn the iteration budget” (L6051–6073)

The table’s analytical content is in the last column. The loop distinguishes three classes of ending: terminal success (no restart machinery involved), terminal failure after recovery is exhausted (retries, nudges, and fallbacks already spent), and non-retryable determinism (the outer except classifier refusing to re-enter). Note that restart *decisions* live mostly outside this table, in the TurnRetryState flags: a compressed-messages restart refunds the iteration and re-enters the same logical step, so it never becomes an exit reason at all. Exit reasons are therefore the residue — what remains after every restart channel has either succeeded silently or hit its one-shot guard. The one-shot convention is load-bearing: a new recovery branch that forgets to set its guard loops forever, and a new restart path that forgets to refund burns budget on recovery rather than progress.

3.3 Budget, Concurrency, Interrupt

3.3.1 IterationBudget: the runaway-spend brake

Two caps are checked in the outer-loop condition: a per-turn counter `api_call_count < agent.max_iterations` (default 90, `run_agent.py:434`) and a thread-safe `IterationBudget` instance (`agent/iteration_budget.py:17–59`). The failure this prevents is unbounded tool-loop churn translating directly into API spend. The core of the counter, `agent/iteration_budget.py:37–45`:

```
def consume(self) -> bool:
    """Try to consume one iteration. Returns True if allowed."""
    with self._lock:
        if self._used >= self.max_total:
            return False
        self._used += 1
        return True
```

Budgets are per-agent, not global: the parent is capped at `max_iterations` (90) and each subagent gets an independent budget capped at `delegation.max_iterations` (default 50), so a delegation tree’s total can exceed the parent’s cap (`iteration_budget.py:20–26`). A shared budget can be injected via the constructor so subagent trees draw from one pool [INFERRED: exact call site not read]. The lock exists because the reflection fork and tool workers share the process.

Two subtleties define the correctness surface. First, **refunds**: `refund()` returns one iteration for `execute_code`-only turns (“cheap RPC-style calls that shouldn’t eat the budget”, `conversation_loop.py:5433–5437`), compression restarts (L4711–4713), redirect/rebuild restarts (L4702–4704, L4733–4738), and runtime-context errors (L1243–1246); each refund also decrements `api_call_count`. Refunds are a manual convention — nothing forces a new restart path to refund, and forgetting one makes recovery consume the budget it exists to protect, terminating the turn prematurely. Second, the **exhaustion path**: when the loop breaks with `budget_exhausted` and no `final_response`, the finalizer makes one extra API call with tools stripped, asking the model to summarize (`turn_finalizer.py:127–142`, [permalink](#)) — budget exhaustion degrades to a summary, not silent truncation; if a verification gate withheld a composed answer, that exact answer is reused instead. A `_budget_grace_call` flag sits in the loop condition but is only ever set to `False` in this snapshot — a vestigial/test seam [INFERRED from exhaustive grep]; a cloner should not replicate it.

3.3.2 Synchronous loop, threaded tool batches

The loop is plain blocking Python — no `async/await` anywhere in the turn path; `finalize_turn`’s docstring states it (“no awaits, no early returns”, `turn_finalizer.py:14–15`). Parallelism is confined to tool batches, which run on a custom pool whose constants sit at `agent/tool_executor.py:95–98` ([permalink](#)):

```
_MAX_TOOL_WORKERS = 8
# Keep this above the stock auxiliary.web_extract timeout (360s) so the
batch
```

```
# guard does not preempt a slow-but-valid summarization attempt.  
_DEFAULT_CONCURRENT_TOOL_TIMEOUT_S = 420.0
```

The 420 s batch wall-clock deadline exists so a slow-but-valid tool cannot wedge a turn indefinitely. The pool is daemon by deliberate construction (`tool_executor.py:700–707`): an abandoned batch is shut down with `wait=False`, but `stdlib ThreadPoolExecutor` workers are non-daemon and joined unconditionally by the `concurrent.futures` `atexit` hook — “so one wedged tool thread would block interpreter exit forever (multi-minute CLI exits).” The failure prevented is a wedged CLI on every interrupted batch; the accepted cost is that a detached tool thread may outlive its turn, which is why request-local cancellation flags exist. Results are appended in original tool-call order; the concurrency verdict (sync loop, daemon pool, 8 workers, 420 s) is corroborated by three independent code reads in the underlying research corpus.

3.3.3 Cooperative three-layer interrupt

Cancellation is cooperative, built from three mechanisms. (1) An agent-level flag, `agent._interrupt_requested`, set by `AIAgent.interrupt()` (`run_agent.py:2839–2940`), which also aborts the active HTTP socket, fans interrupt bits out to tool-worker thread IDs, and recursively interrupts child subagents. (2) Per-thread interrupt bits in `tools/interrupt.py:34–70` — a process-global set keyed by thread ID, so interrupting one session “does not kill tools running in other sessions ... critical in the gateway where multiple agents run concurrently in the same process.” (3) The interruptible API call itself (`agent/chat_completion_helpers.py:506–518`), which runs the HTTP request on a daemon worker while the main thread polls; on interrupt only the worker’s sockets are shut down and the resulting transport error is swallowed via a request-local flag. A `/stop` therefore unwinds deterministically: socket abort — swallowed worker error — `close_interrupted_tool_sequence` patches dangling `tool_calls` — break with `interrupted_during_api_call` — finalizer closes the transcript tail. The streaming side is protected by a single-writer fence: each attempt claims a monotonic token before consuming its stream (`run_agent.py:5341–5358`), and superseded writers’ deltas are dropped — the failure prevented is two overlapping attempts interleaving text into the consumer. The fence degrades to “no fence” rather than raising, after a real cron crash on a missing attribute (`agent/stream_single_writer.py:14–21`).

3.4 Errors and Honest Assessment

3.4.1 Error classification and the provider seam

Every API failure in the inner loop passes through `classify_api_error(...)` (`agent/error_classifier.py:24–98`; call site `conversation_loop.py:3032–3044`), which returns a `FailoverReason` plus recovery hints — `retryable`, `should_compress`, `should_rotate_credential`, `should_fallback`. The loop consumes the hints: jittered decorrelated backoff to avoid thundering-herd retries against a shared rate-limited provider; `Retry-After` honored with a 600 s cap; context-overflow errors converted into compression restarts rather than blind retries; `should_fallback` errors routed to `_try_activate_fallback`, which swaps providers mid-conversation and resets the retry counters. This chapter owns the seam, not the taxonomy: the ~25-reason `FailoverReason` enum and the five-layer provider architecture it drives are chapter 9’s subject.

3.4.2 The 3,900-line loop as design risk

The god-file decomposition moved the pressure; it did not eliminate it. `run_conversation` alone is ~3,900 lines, its inner retry region on the order of 2,400 — a museum of incident IDs (#26293, #29507, #32421, #65991, #66267), each a real provider failure converted into a guarded recovery branch. The load-bearing regions are small and extractable: the outer while-condition, the consume/refund discipline, the tool-dispatch branch, the finalizer’s summary fallback. The incidental regions — provider-specific recoveries, MoA injection, redirect/steer plumbing — are additive robustness a clone can defer. The blast radius of a careless edit inside the retry region remains the runtime’s largest single design risk: a branch that violates the one-shot guard or refund convention degrades every turn silently rather than failing loudly.

3.4.3 Clone notes

Clone notes. Milestone one of a clone is the loop plus the provider path, in that order of risk: a minimal `run_conversation` equivalent — outer budget while, inner retry while, `IterationBudget` with the summary-call fallback, sequential tool dispatch — plus one streaming-capable `chat_completions` transport. That core is the largest single slice of the 10–15% minimal core described in Chapter 10, Part B. Defer the streaming fence, compression restarts, failover chain, and redirect machinery; each is additive and each has a named failure it prevents, so the deferral cost is knowable. Preserve two conventions verbatim from day one: the OpenAI-shaped internal message representation (chapter 9’s invariant) and the refund-on-restart discipline — the second is the one a fresh implementation is most likely to get wrong, because nothing in the type system enforces it.

5. Context Engineering and Prompt-Cache Economics

The organizing claim of this chapter is insight 1 of this study: prompt-cache economics is the hidden architect of Hermes Agent. Five mechanisms in otherwise unrelated subsystems — storage schema, prompt assembly, memory injection, the background reflection fork, and async completion delivery — exist for one reason: to keep the byte prefix of every API request identical to the previous request’s, so provider KV caches hit and the operator is not re-billed for re-prefilling thousands of tokens per turn. This is a load-bearing constraint, not an optimization. What follows specifies the doctrine, the machinery that bounds what it protects, and a cost model for how much of it to keep.

5.1 The Byte-Stability Doctrine

Anthropic-style prompt caches and OpenAI-style implicit prefix caches both match on leading bytes; any early mutation invalidates everything after it. Hermes prices a full prefix miss on Anthropic routes at roughly a 75% input-token cost delta, plus re-prefill latency. Hence the doctrine: order every request stable bytes first, volatile bytes last — then never touch the leading bytes for the life of a session.

5.1.1 Three-tier prompt assembly

The system prompt is assembled once per session as three tiers joined with `\n\n` — `stable` → `context` → `volatile` — cached on `agent._cached_system_prompt`, and rebuilt only after compression (`agent/system_prompt.py:10-19`, [permalink](#)). Stable carries identity (`SOUL.md` or the default), tool guidance, the skills index, and environment/platform hints probed once and never re-probed. Context carries the caller-supplied `system_message` plus at most one project context file (`.hermes.md`, `AGENTS.md`, `CLAUDE.md`, or `.cursorrules`, first match wins). Volatile carries the frozen memory snapshot, user profile, external memory-provider block, and the timestamp line. The in-code contract states that Hermes never reinjects parts of the block mid-session, “which is the only way to keep upstream prompt caches warm across turns” (`agent/system_prompt.py:535-540`).

Ordering does the work. A volatile-tier change (a memory write, a new day) invalidates only the cache tail; a stable-tier change would invalidate everything, so mid-session additions go into the user message or tool results, never the prompt. Session resume likewise restores the persisted prompt verbatim (`agent/conversation_loop.py:434-464`).

The timestamp line is the doctrine in miniature: minute precision would change the prompt on every rebuild path and invalidate prefix-cache KV each time. Hermes renders the date only, `agent/system_prompt.py:503-511`:

```
now = _hermes_now()
# Date-only (not minute-precision) so the system prompt is byte-stable
# for the full day. Minute-precision changes invalidate prefix-cache
KV
# on every rebuild path (compression boundary, fresh-agent gateway
turns,
# session resume without a stored prompt). The model can still query
the
# exact wall-clock time via tools when it actually needs it.
# Credit: @iamfoz (PR #20451).
timestamp_line = f"Conversation started: {now.strftime('%A, %B %d,
%Y')}"
```

The trade-off is deliberate information loss — the model must call a tool for wall-clock time — against up to 24 hours of byte-stability per rebuild path: the smallest, most liftable instance of the doctrine.

5.1.2 Byte-identical replay: the `api_content` sidecar

The doctrine’s hardest problem is per-turn injection. Hermes appends ephemeral context — `pre_llm_call` plugin output, external-memory prefetch — to the current turn’s user message, not the system prompt. On the next turn that message is history: replay it without the injection and the prefix diverges at exactly that point; persist the injection and it leaks into search, trajectories, and future sessions.

The resolution is a sidecar. The composed wire bytes are stamped onto the message as `api_content` and persisted to the session SQLite store alongside the clean content; one helper

produces both, so the sidecar can never drift from the bytes on the wire — the invariant being “what turn N sends must be what turn N+1 replays” (`agent/turn_context.py:50-66`, [permalink](#)). Later turns substitute the sidecar when building `api_messages` (`agent/conversation_loop.py:1022-1036`, [permalink](#)); assistant rows can carry an analogous sanitize-divergence sidecar. Three dimension reports verified the mechanism independently across loop, prompt, and storage layers.

The trade-off is byte-fidelity replay versus storage bloat: every injected message is stored twice, once clean and once as sent. That is cheap in SQLite; the alternative — re-deriving injections at replay time — would couple replay to every plugin, so any plugin behavior change would silently break replay. Hermes pays the storage.

5.1.3 Anthropic *system_and_3*

Byte-stability keeps the prefix cacheable; Anthropic’s explicit cache-control API additionally requires marking where the boundaries are, at a maximum of four breakpoints per request. `system_and_3` places up to four: one on the system prompt, one on each of the last three eligible non-system messages, all at one TTL (5m default, 1h optional). The selection logic, `agent/prompt_caching.py:109-116`:

```
remaining = 4 - breakpoints_used
non_sys = [
    i
    for i in range(len(messages))
    if messages[i].get("role") != "system"
    and _can_carry_marker(messages[i],
native_anthropic=native_anthropic)
]
for idx in non_sys[-remaining:]:
```

([permalink](#)). The functions are pure transforms — input deep-copied, annotated, returned — so stored history is never decorated. The eligibility predicate `_can_carry_marker` encodes provider reality: on the envelope/OpenAI-wire layout (OpenRouter, Nous Portal, Alibaba), markers on empty-content messages are silently ignored or hang the route, so they are skipped (`agent/prompt_caching.py:52-73`). A provider-policy matrix (`anthropic_prompt_cache_policy`, `agent/agent_runtime_helpers.py:1813-1888`) decides per route whether caching applies and in which layout; the measured justification is that including Kimi on OpenRouter moved cache hits from 1% to 97% within a turn (#25970).

The rolling three-message tail is what lets the strategy survive history churn: tail breakpoints move with the frontier as tool turns append messages, the system-prompt breakpoint anchors the large static block, and after compression invalidates the middle, caching re-establishes within one or two turns. OpenAI-style routes get implicit prefix caching for free from byte-stability alone [INFERRED — no explicit marker path for non-Anthropic routes exists]. The cost to note: model and credential identity participate in the cache key, so a mid-session model switch or credential rotation zeroes the cache.

5.2 Compression and Budgeting

Byte-stability determines what must not change; compression and budgeting determine how much of it there is. Together they bound growth of the cached prefix from two directions: total tokens (compression) and per-turn tool output (the three-layer budget).

5.2.1 Dual triggers, four phases, and the engine contract

Compression fires from two layers. A gateway-side hygiene pass triggers at 85% of the context window before the agent sees an inbound message — a safety net deliberately set above the in-loop threshold so long sessions do not compress every turn (gateway/run.py:12832-12861). The in-loop ContextCompressor defaults to 50% of the **main model's** window (never the auxiliary summarizer's), with per-model overrides and a raise-only 0.75 floor under 512K windows against thrash. Three in-loop sites check pressure: a turn-prologue preflight, a pre-API check on the fully assembled request (catching turns that grew via huge tool results), and a post-response check preferring API-reported prompt_tokens — completion and reasoning tokens do not consume window (#12026).

compress() runs four phases (agent/context_compressor.py:4256-4650): (1) prune old tool results over 200 chars outside the protected tail to a placeholder, at no LLM cost; (2) compute boundaries — head is the system prompt plus the first three non-system messages, tail is cut by token budget (20% of threshold) walking backward, both aligned so tool_call/tool_result groups are never split; (3) summarize the middle with the auxiliary client — or, on re-compression, *update* the previous summary with new turns, rehydrated from persisted “fossil” messages after a resume; (4) assemble head + summary + untouched tail, sanitize orphaned tool pairs, and invalidate the cached prompt. Iterative summarization is the main defense against summary drift: each compaction edits an accumulating document instead of re-compressing a moving window. The loop side — the restart that refunds the consumed iteration — is covered in ch3.

The compressor sits behind a plugin contract. ContextEngine (agent/context_engine.py:89-351) requires a name, update_from_response(usage), should_compress(prompt_tokens), and compress(...), plus mandatory token-accounting attributes; lifecycle hooks and engine-exposed tools are optional with safe defaults. Selection is config-driven (config setting → repo-shipped plugins → general plugins → built-in fallback); plugin engines are never auto-activated. The contract lets a lossless engine — one that pages context to disk rather than summarizing — replace the summarizer without host changes.

5.2.2 The three-layer tool-result budget

Compression is reactive; the tool-result budget is preventive, and the most production-transferable pattern in this chapter. The doctrine is stated verbatim at the top of tools/tool_result_storage.py:3-10:

Defense against context-window overflow operates at three levels:

1. ****Per-tool output cap**** (inside each tool): Tools like search_files pre-truncate their own output before returning. This **is** the first line of defense **and** the only one the tool author controls.

2. ****Per-result persistence**** (`maybe_persist_tool_result`): After a tool returns, **if** its output exceeds the tool's registered threshold

([permlink](#)). Layer 1 is decentralized: each tool pre-truncates against config-tunable caps (terminal at 50,000 chars with a 40/60 head/tail split; `read_file` at 2,000 lines). Layer 2, `maybe_persist_tool_result`, fires when a result exceeds its registered threshold: the full output is written *into the sandbox* temp dir — via a stdin pipe, dodging the 128 KB `MAX_ARG_STRLEN` exec-arg ceiling so the spill is reachable on any backend — and the in-context content becomes a $\leq 1,500$ -char preview in `<persisted-output>` tags with the path; the model recovers the full text with `read_file` when needed. Layer 3, `enforce_turn_budget`, catches the aggregate case: if one assistant turn's results exceed 200K chars, the largest non-persisted results are force-spilled until the total fits.

Budgets scale to the model window. `budget_for_context_window()` (`tools/budget_config.py`:84-114, [permlink](#)) computes per-result and per-turn budgets as 15% and 30% of `context_length × 4` chars, clamped so large models keep the historical 100K/200K defaults as caps and small models floor at 8K/16K — the fix for a 65K-window model blowing its context on one persisted result (#23767). The one pinned threshold is the load-bearing detail: `read_file` is pinned to infinite (`tools/budget_config.py`:10-13). If `read_file` output could itself be spilled, the model would spill a file, read it back, spill the read, and loop; the pin exempts the recovery tool from the mechanism it recovers from. Two dimension reports independently verified the three layers.

5.3 The Cost Model

5.3.1 The cache-stability mechanism map

The doctrine's footprint spans five mechanisms in five subsystems. The table maps each to its host subsystem and the chapter covering it in depth; this chapter covers the first two and cross-references the rest.

Mechanism	Host subsystem	In-depth chapter	What it protects
api_content sidecar	Session storage / loop replay (agent/turn_context.py:50-66)	\$5.1.2	Byte-identical replay of injected messages
Date-only timestamps	Prompt assembly (agent/system_prompt.py:503-511)	\$5.1.1	Day-long prompt byte-stability
Frozen memory snapshot	Memory subsystem (build-time freeze)	ch6	Volatile-tier immutability within a session
Cache-parity fork	Learning loop (background)	ch7	Child agent reuses parent's warm cache

Mechanism	Host subsystem	In-depth chapter	What it protects
	reflection)		prefix
Idle-only completion delivery	Delegation / async events	ch8	Role-alternation invariant and tail-byte stability

Read as a set, the five rows make insight 1's case better than any single mechanism can. A storage schema, a timestamp format, a memory refresh policy, a process-fork design, and a message-delivery schedule share no code, no module, and no rationale except one: keep the provider prefix byte-stable so caches hit. Remove the constraint and each mechanism looks like an oddity; restore it and all five are the same decision at different layers. For a builder of any agent, the map doubles as a checklist: whichever of these subsystems you build, the cache-stability question will arrive inside it, and it is cheaper to answer at design time than to retrofit. The trade-offs are real in every row — duplicate storage, lost clock precision, stale facts, fork complexity, delayed notifications — and Hermes pays each one, because the alternative is full prefill cost on every turn.

5.3.2 Inverse payoff

The cost model runs both directions. Ignoring cache stability does not break a clone functionally, but it materially raises per-turn cost and sacrifices prefill latency on every provider that offers caching. Conversely, the apparatus is only worth its complexity where a cache exists to hit: a clone targeting providers without prefix caching can delete roughly 15–20% of the system's complexity, as the sidecars, replay machinery, frozen-snapshot discipline, cache-parity fork, and breakpoint placement all collapse into “send the history.”

Clone notes. Lift in this order: (1) date-only timestamps and tiered assembly — one day of byte-stability for one line; (2) the three-layer tool-result budget with the `read_file: infpin` — provider-independent overflow defense that preserves capability via spill-and-preview instead of truncating it; (3) the `ContextEngine` contract so compression policy stays swappable; (4) the `api_content` sidecar only once you route to a prefix-caching provider — until then it is pure storage overhead. The inverse payoff, stated plainly: a provider without prefix caching lets you delete ~15–20% of what this chapter describes. Byte-stability is a bet on your provider's cache; size the bet accordingly.

6. Memory and Skills: The Self-Authored Data Layer

Hermes keeps one conversation loop and pushes complexity into data the loop reads (chapter 5 established the token economics). This chapter covers the two data layers the agent curates for itself — bounded Markdown files holding facts and user preferences, and `SKILL.md` packages holding procedures — plus the SQLite session store beneath both, which provides recall with no LLM in the path. The unifying bet is that an agent's long-term state should be a diffable, greppable artifact a human can audit with `git diff` and `grep`; the chapter is equally about what that bet forfeits: semantic-similarity recall and scale beyond what fits in a prompt.

6.1 Agent-Curated File Memory

6.1.1 Two bounded files, edited by the agent

Stale-source note (C5). Older articles place `memory.md` and `user.md` at the config root. The authoritative location is `$HERMES_HOME/memories/MEMORY.md` and `USER.md`, resolved dynamically per profile by `get_memory_dir()` ([tools/memory_tool.py:55–57](#)) — the code comment notes the older module-level constant “could go stale if a profile switch happened after the first import.” Code at HEAD, snapshot July 2026.

The two stores have distinct subjects. `MEMORY.md` is the agent’s notebook — environment facts, project conventions, tool quirks — default budget 2,200 characters; `USER.md` holds what the agent knows about the user — preferences, style, expectations — default 1,375 ([tools/memory_tool.py:5–14](#)). Entries are delimited by `\n`, may be multiline, and budgets are character-based because “char counts are model-independent” ([tools/memory_tool.py:69](#); overrides at [agent/agent_init.py:1601–1604](#)).

The write mechanics assume a fallible, concurrent world. Replace and remove target a short unique substring rather than an entry ID; multiple matches return an error with previews ([tools/memory_tool.py:398–399](#)). There is deliberately no read action — the content is always in the system prompt, so a read would only invite thrash. Every mutation runs under an exclusive file lock with atomic temp-file-plus-rename writes ([tools/memory_tool.py:253–288](#), [769–798](#)) and is scanned for injection/exfiltration patterns before it lands ([:88–90](#)). Before any replace, remove, or batch, the on-disk file is round-trip checked; externally appended content (a shell append, a sister session) aborts the mutation and is preserved to `.bak`. [<ts> \(:714–767, issue #26045\)](#). Over-budget writes trigger a self-consolidation loop capped at three consecutive failures per turn ([:138](#), [379–390](#)), so a fragile write cannot consume the whole turn.

The architectural statement is worth naming plainly: long-term memory is a bounded text file the agent edits, and every property of the design — substring targeting, character budgets, `$`-delimiters — exists to keep that file human-auditable and machine-editable at once. What is forfeited is equally plain: no embedding index, no similarity recall, and a 2,200-character ceiling that holds only what the agent judged worth compressing into a few dozen lines. The ceiling is not a limitation being worked around; it is the point — the whole store must be cheap enough to live permanently in the prompt.

6.1.2 The frozen-snapshot invariant

`MemoryStore` maintains two parallel states ([tools/memory_tool.py:123–132](#)): `_system_prompt_snapshot`, captured once at `load_from_disk()` and never mutated mid-session, and live `memory_entries/user_entries` that tool calls mutate and persist to disk immediately. The split is stated in the code’s own words at [tools/memory_tool.py:626–632](#):

```
"""
```

```
Return the frozen snapshot for system prompt injection.
```

```
This returns the state captured at load_from_disk() time, NOT the live state. Mid-session writes do not affect this. This keeps the system
```

prompt stable across all turns, preserving the prefix cache.
"""

The snapshot is injected exactly once, during system-prompt assembly, into the volatile tier (agent/system_prompt.py:483–492). Writes made at turn 40 hit disk immediately — the next session, and any tool response in this one, sees them — but the running session’s prompt keeps the bytes it started with. The rationale is chapter 5’s: a mid-session prompt mutation would invalidate the provider prefix cache on every subsequent turn, converting a two-kilobyte edit into a full-prefix recompute per call. Freshness is what is deliberately traded away.

Insight 1 (cache parity). *The frozen snapshot is one instance of a system-wide constraint: five unrelated subsystems exist partly to keep the prompt prefix byte-stable. A clone targeting a provider without prefix caching can delete this machinery; a clone that ignores it pays for the deletion in per-turn cost.*

6.2 Session Store and Retrieval

6.2.1 One state.db per profile, three FTS5 indexes

Each profile owns one SQLite state.db (hermes_state.py:154) in WAL mode, holding sessions (metadata, lineage via parent_session_id, token and cost counters, a snapshot of the assembled system prompt) and messages (full history, including the api_content sidecar chapter 5 covers). Full-text recall runs over three FTS5 indexes: messages_fts (unicode6l, external-content, trigger-synced); messages_fts_trigram for substring queries, built over a view excluding role='tool' rows — roughly 90% of stored bytes are machine noise, so the index stays about 2.6× smaller; and messages_fts_cjk, a loadable tokenizer emitting CJK bigrams so one-to-two-character CJK queries hit index speed instead of multi-second LIKE scans (hermes_state.py:1187–1392).

The deepest detail is schema v23’s marker-gated online reindexing: while a chunked background rebuild runs, two state_meta keys (high-water H, progress P) define which rows are indexed, and every sync trigger’s WHEN clause gates on that predicate, because firing an external-content delete for an unindexed row is the canonical FTS5 corruption hazard. The base index definition is at hermes_state.py:1187–1194:

```
FTS_SQL = """
CREATE VIRTUAL TABLE IF NOT EXISTS messages_fts USING fts5(
    content,
    tool_name,
    tool_calls,
    content='messages',
    content_rowid='id'
);
```

Queries are BM25-ranked with optional temporal sort, source and role filters, and rewind exclusion, routed by script across the three indexes (hermes_state.py:7101–7300).

6.2.2 session_search: recall with no LLM in the path

Stale-source note (C3). The README still claims “FTS5 session search with LLM summarization.” The summary path was removed; the module docstring at [tools/session_search_tool.py:1–29](#) is explicit. Code at HEAD, snapshot July 2026, is authoritative; lines 21–28:

All three modes operate on the SQLite session DB via the FTS5 index **and** the `get_anchored_view` / `get_messages_around` primitives **in** `hermes_state`. No LLM calls anywhere – every shape returns actual messages **from** the DB.

History: PR #20238 (*JabberELF*) seeded a fast/summary dual-mode split; the toolkit expansion **in** PR #26419 (*yoniebans*) added the anchored drill-down, bookends, **and** sort. This module merges all of that into a single calling shape **with** no mode parameter, no summary LLM path, **and** explicit scroll

The tool offers four calling shapes inferred from arguments (the docstring predates the fourth, a READ shape at schema line 955): DISCOVERY — an FTS scan deduplicated by session lineage, returning per-session snippets, a ± 5 -message window, and first/last-three-message “bookends,” so the model gets goal → match → resolution without paying for a whole transcript; SCROLL — windowed paging around an anchor; READ — a whole session, head-and-tailed when large; BROWSE — recent sessions with titles. Results return as an ordinary tool-result message; nothing is injected automatically. The consequence: cross-session recall is deterministic, inference-free, and inspectable — you can run the same SQL the tool ran.

6.2.3 The MemoryProvider ABC and one-provider exclusivity

Above the built-in layers sits a plugin interface, `MemoryProvider` (`agent/memory_provider.py:43`), with lifecycle methods (`initialize`, `system_prompt_block`, `prefetch`, `sync_turn`, `shutdown`) and optional hooks (`on_pre_compress`; `on_memory_write`, which mirrors built-in file writes to the external backend). Eight providers are bundled — `honcho`, `mem0`, `hindsight`, `supermemory`, `byterover`, `holographic`, `openviking`, `retaindb` — and the manager enforces a hard rule: exactly one external provider may be active; a second registration is rejected “to prevent tool schema bloat and conflicting memory backends” ([agent/memory_manager.py:394–416](#)). The exclusivity buys a single retrieval path with no federation ambiguity. Prefetched recall is fenced as `<memory-context>` and scrubbed if it leaks into output; a wedged provider is isolated behind an 8-second prefetch timeout and a single-worker sync executor. [INFERRED] The “builtin” provider slot reserved in `MemoryManager` suggests a planned first-party provider, but at the snapshot the built-in files are wired directly via `_memory_store`, so the registry holds zero or one external provider in practice.

The three layers, mapped onto the CoALA memory taxonomy the rest of this study uses:

Layer / store	Mutability	Injection timing	Retrieval path	Budget / cap
File memory (MEMORY.md, USER.md)	Agent-writable, any turn; snapshot frozen	Once per session, system prompt	Always present (no retrieval)	2,200 / 1,375 chars

Layer / store	Mutability	Injection timing	Retrieval path	Budget / cap
	per session			
Session store (state.db + FTS5)	Append-only by harness; soft- delete on rewind	Never automatic; only via tool result	session_search tool, BM25 over three indexes	Unbounded history; ~300- row discovery scan
External provider (one max)	Provider- defined; mirrored built- in writes	Per-turn prefetch, fenced into API copy	Provider tools and/or auto- injected context block	Provider- defined; 8 s prefetch timeout

The triad partitions the CoALA space cleanly. File memory is the semantic store — facts and preferences distilled into durable form — with the frozen snapshot making its injection cost zero after session start. The session store is episodic memory: raw episodes, paid for only when the model asks, which is why the retrieval path can afford to be an ordinary tool call rather than an embedding pipeline. The provider slot is the escape hatch for deployments needing semantic recall at scale, and the one-provider rule acknowledges that adding it is an architectural commitment, not an additive feature. Working memory (the assembled prompt and in-context history) is chapter 5’s subject. Procedural memory is deliberately absent from the table; in Hermes it is not a memory layer at all but a separate artifact class — the subject of the next section.

6.3 Skills as Procedural Memory

6.3.1 *SKILL.md: an interop bet, not a bespoke format*

A skill is a directory containing a `SKILL.md` plus optional support directories (`references/`, `templates/`, `scripts/`, `assets/`) that are progressive-disclosure data, never discovery roots ([agent/skill_utils.py:50](#)). The front matter is declared “agentskills.io compatible” (`tools/skills_tool.py:28–46`): required name (≤ 64 chars) and description (≤ 1024 chars, truncated to ~ 60 in the index), the rest optional. Hermes extensions are namespaced under `metadata.hermes.*` — `requires_toolsets` for conditional activation, `config` for non-secret settings, blueprint for cron-automation packaging — keeping the top level standard-compliant. The format choice is an interop bet: skills written for Hermes should be readable by any agent honoring the shared standard, and others’ skills (the Hub, §6.3.4) install without translation. The trade is that Hermes’s richer semantics live in a vendor namespace the standard ignores.

6.3.2 *Three-tier progressive disclosure*

The system prompt embeds only a compact index — per category, - name: description lines inside `<available_skills>`, wrapped in a mandatory-load preamble at [agent/prompt_builder.py:1732–1758](#): “If a skill matches or is even partially relevant to your task, you MUST load it with `skill_view(name)`... Err on the side of loading.” The index sits in the stable prompt tier (`agent/system_prompt.py:316–324`), so it rides the prefix cache for the whole session; nothing is ever fully hidden, since `compact_categories` demotes categories to names-only lines under prompt-size pressure rather than removing them.

Tier	What is loaded	When	Token cost	Invalidation
0 — index	Category + name: description lines, ~60-char descriptions	Every session, stable prompt tier	~3k tokens for ~120 skills	LRU + disk snapshot, mtime/size manifest, 30 s scan signature
1 — body	Full SKILL.md body + metadata, injected as a tool result	On skill_view(name) — mandatory per preamble on partial relevance	Body size; house style ~100–200 lines	None mid-session; preamble governs re-loads
2 — support files	One file from references/ templates/ scripts/ assets/	On skill_view(name, file_path=...)	Single file only	None; loaded fresh per call

The economics mirror the memory triad’s logic at one remove. Tier 0 is the standing cost: a few thousand tokens, paid once per session and cache-amortized, buying just enough signal to route — the description *is* the trigger, there is no separate triggers field, which is why the house style’s 60-character limit is enforced as “NOT cosmetic.” Tiers 1 and 2 make procedure pay-per-use: a two-hundred-line workflow enters context only when a task plausibly needs it, and support files — the long tail of reference material — never enter unless explicitly fetched. The forfeiture is that routing quality rests entirely on short descriptions and a prompt-level mandate; there is no embedding retrieval over skill bodies, so a badly described skill is a skill that silently never loads.

6.3.3 *skill_manage: writes with provenance*

The agent edits its procedural memory through `skill_manage` — `create`, `patch` (old/new string, preferred), `edit`, `delete`, `write_file`, `remove_file` — with guards on every write: front-matter and size validation, a 100 KB cap on agent-authored content (`tools/skill_manager_tool.py:488`), read-before-write inside the background review, and an optional `skills.write_approval` staging gate. The load-bearing guard is provenance. A `ContextVar` records the write origin — `tools/skill_provenance.py:37–45` (eliding comments):

```
_write_origin: contextvars.ContextVar[str] = contextvars.ContextVar(
    "skill_write_origin",
    default="foreground",
)
BACKGROUND_REVIEW = "background_review"
```

Only skills created inside the background reflection fork are marked `agent_created`; foreground, user-directed creates belong to the user and are curator-exempt (`tools/skill_manager_tool.py:1421–1427`). Provenance answers the question every self-modifying system must answer first: who wrote this procedure — a human, the hub, or the agent itself? Without the flag, autonomous maintenance would eventually prune the user’s own skills; with it, the decay lifecycle (chapter 7’s Curator) applies only to agent sediment. The write-trigger

cadences — a skill nudge every ten tool iterations firing a whitelisted background fork — and the Curator’s active → stale(30 d) → archive(90 d) transitions are chapter 7’s subject, beyond one note here: archive-never-delete is a debugging affordance, not sentimentality — a rotted procedure can be diffed against the version that worked.

6.3.4 Skills Hub: untrusted text through a scanning funnel

The Hub is nine SkillSource adapters — GitHub taps (openai/skills, anthropics/skills, and others), optional-skills, skills.sh, well-known indexes, URL, ClawHub, Claude Marketplace, LobeHub, Browse.sh — behind one ABC ([tools/skills_hub.py:474–507](#); adapter count is a single-source code read). Installs follow one pipeline: fetch → quarantine under `~/.hermes/skills/.hub/quarantine/` → regex scan for exfiltration, prompt injection, destructive commands, and persistence (`tools/skills_guard.py`) → trust-tier policy (builtin never scanned; community blocks any findings unless forced; dangerous never overridable) → install with lockfile and audit log.

Insight 5 (the outer ring). *Hermes’s extensibility is concentric — registry → toolsets → plugins → MCP → skills — capability, cost, and trust decreasing together moving outward. Skills are the outermost ring and the only layer the agent itself writes: untrusted-but-scanned text in, provenance-tracked text out. The learning loop therefore requires only the cheapest ring to exist.*

6.3.5 Clone notes

Clone notes. *Memory is milestone six of the staged roadmap (~200 LOC: two Markdown files, \$-delimited entries, char budgets, one substring-targeted memory tool with atomic writes, frozen-snapshot injection at session start). Skills are milestone seven (~300 LOC: a ~150-line front-matter parser, one discovery root, `skills_list/skill_view/skill_manage`, index injection into the stable tier). Both are individually omissible — the loop runs a full turn without either — and both depend only on the context pipeline, not on each other. Carry the provenance flag from day one; retrofit it after autonomous writes exist and you can no longer tell agent sediment from user property. Defer the trigram/CJK indexes, rebuild markers, Hub, and Curator until scale or untrusted intake actually arrives.*

7. The Learning Loop: Self-Improvement as Disciplined Text Gardening

This is the chapter the repository’s reputation rests on, so it opens by correcting expectations. Hermes Agent’s “self-improvement” is real, closed-loop, and on by default — and narrower than the phrase suggests. The system turns conversational experience into bounded, human-readable Markdown artifacts (skills, memory entries, a user model), injects them into future prompts, and gardens them over time; it never mutates code, weights, or its own runtime prompts. The narrowness is the safety story, not a gap.

7.1 The Honest Frame

7.1.1 Self-organizing, not self-modifying

The organizing claim is insight 3, and the taxonomy dimension’s verdict states it verbatim: **self-organizing, not self-modifying**. Three absences define the mechanism, and each is load-bearing:

- **No in-loop fitness measurement.** Nothing in the main repository scores whether a learned skill improved anything. Quality is prompt-mediated — it rests on the reviewing model’s judgment plus the review prompt’s anti-capture rules — and so degrades gracefully with model quality rather than failing loudly.
- **No weight updates.** There is no training path in the runtime; the RL layer that once existed was amputated in one PR in May 2026 (chapter 9). “Learning” here means the filesystem changes.
- **No runtime prompt self-modification.** The agent cannot rewrite its system prompt, tool descriptions, or review prompts. Learned artifacts re-enter context only through sanctioned injection points — the skills index and the memory snapshot — and only at session boundaries, because mid-session mutation would break chapter 5’s prefix-cache invariant.

What remains as the mutation surface is deliberately small: whitelisted, bounded, human-readable Markdown writes — `SKILL.md` packages under `~/hermes/skills/`, `MEMORY.md` and `USER.md` under `$HERMES_HOME/memories/` — capped at 100 KB per skill and 1 MiB per file, written through tools carrying provenance and ownership guards ([tools/skill_manager_tool.py:488–489](#)). Safety is achieved by *constriction*: the system does not police a general self-modification surface, it declines to have one — insight 5’s outermost ring made behavioral. The absences are why the loop may run unattended by default; a system that measured fitness and rewrote itself in-loop would need exactly the human-review gates this project reserves for its external companion (§7.4.1).

7.2 The Background Reflection Fork

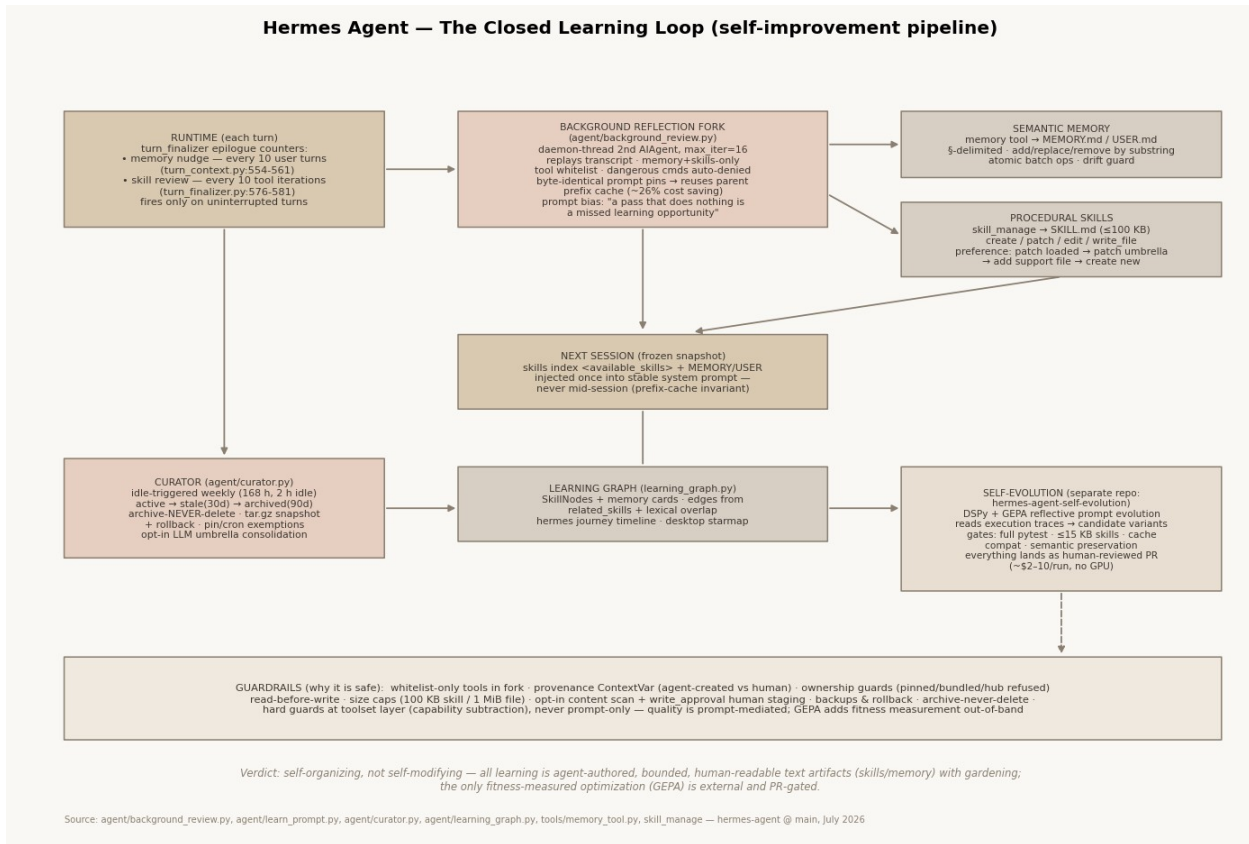


Figure 3: The closed learning loop — epilogue cadences fire a cache-parity reflection fork whose artifacts re-enter the next session’s prompt; the Curator gardens them; fitness-measured evolution is external and PR-gated.

Figure 3 traces the circuit: two cadence counters fire a forked second agent after response delivery; the fork writes memory and skill artifacts through a whitelisted tool surface; the artifacts re-enter the next session as frozen prompt data; the Curator decays them; and the only fitness-measured optimization happens outside the repository, landing as human-reviewed PRs.

7.2.1 Cadences: two counters, one fire condition

Reflection is scheduled by two counters, not a timer. The **memory nudge** counts user turns: `_turns_since_memory` increments per turn context and fires at `_memory_nudge_interval` (default **10 user turns**, `memory.nudge_interval`), rehydrated from persisted history across restarts ([agent/turn_context.py:554–561](#); rehydration :520–526). The **skill nudge** counts tool-calling iterations: `_iters_since_skill` increments per iteration ([agent/conversation_loop.py:889–891](#)) and is checked in the finalizer against `_skill_nudge_interval` (default **10 tool iterations**, `skills.creation_nudge_interval`), resetting on fire ([agent/turn_finalizer.py:576–581](#)). The metrics are complementary: turns approximate elapsed experience; iterations approximate *difficulty* — ten tool calls means a non-trivial procedure was likely just discovered.

The fire condition matters as much as the cadences: the review spawns only when `final_response` and not interrupted and (`review_memory` or `review_skills`) — **after response delivery, only on successful uninterrupted turns** (`agent/turn_finalizer.py:591-601`), per the code comment, “so it never competes with the user’s task for model attention.” An interrupted turn produces no learning, which quietly keeps error transcripts out of the skill mine. The spawn is best-effort — a failed review can never fail a turn — and lands on a daemon thread named `bg-review` (`run_agent.py:1688-1714`), so a wedged review cannot hold the process open.

7.2.2 The fork: a second AI Agent with its capabilities subtracted

`_run_review_in_thread` (`agent/background_review.py:617-953`) constructs a **full second AI Agent**, not a one-shot prompt. The weight is the point: the reviewer can `skill_view` the exact skill it intends to patch before writing, which is what makes the read-before-write guard (§7.4.2) enforceable at all. The fork inherits the parent’s live runtime — provider, model, base URL, credentials — because re-running credential resolution fails for OAuth-only and session-scoped setups (`background_review.py:659-665`). Then the hardening begins, all of it capability subtraction (insight 4), never instruction:

- A **thread-scoped tool whitelist** restricts the fork to memory and skill tools; everything else is denied at dispatch (`background_review.py:819-835`).
- **Dangerous-command approvals auto-deny** on the worker thread — a blocking `input()` would deadlock against the parent’s TUI (#15216), at `background_review.py:637-644`:

```
def _bg_review_auto_deny(command, description, **kwargs):
    logger.warning(
        "Background review auto-denied dangerous command: %s (%s)",
        command, description,
    )
    return "deny"
try:
    _set_approval_callback(_bg_review_auto_deny)
```

- **_persist_disabled = True** hard-stops every session-DB write path (`background_review.py:743-754`). The comment names this “the curator-takeover root cause”: because the fork shares the parent’s `session_id` (for cache warmth, §7.2.3), without persistence isolation its review prompt would land in the user’s real session, and the next live turn would re-read it as a standing instruction and “become” the curator. Session-identity sharing for cache economics opened a prompt-injection channel from the agent’s own reflection machinery; the fix removed the write capability rather than filtering the text.
- **Compression is disabled** (`background_review.py:796-807`): a compression race won by the single-lifecycle fork would rotate the parent’s session into a child the gateway never adopts (#38727). The fork’s own nudges are zeroed — no recursive reviews — and `stdout` is silenced thread-locally (#55769).

The review prompts complete the design. The memory prompt is conservative — save persona, preferences, expectations, else “say ‘Nothing to save.’ and stop” (`background_review.py:170-`

179). The skill prompt is deliberately bias-to-act: “Be ACTIVE — most sessions produce at least one skill update... A pass that does nothing is a missed learning opportunity, not a neutral outcome” (background_review.py:181–369). The system *wants* sediment and then pays the Curator to fight the bloat (§7.3.2). Counterweighting the bias is an anti-capture list (background_review.py:260–275): never capture environment-dependent failures, transient errors, one-off narratives, or negative tool claims, because “these harden into refusals the agent cites against itself for months.” That sentence explains why prompt-mediated learning needs editorial rules: a learned artifact is a future prompt, and a false negative claim in it is a self-inflicted capability loss.

7.2.3 Cache-parity pins: reflection priced at ~26% less

A full agent loop (up to 16 iterations) every ten turns would naively double token spend on those turns [INFERRED from the ~26% figure cited at background_review.py:772–774]. It does not, because the fork is engineered to hit the parent’s warm provider prefix cache — insight 1 appearing inside the learning loop. On the same-model path the fork inherits the parent’s cached system prompt verbatim and pins every other value that could perturb the byte prefix, at background_review.py:779–789 (defensive comment elided):

```
if not _routed:
    review_agent._cached_system_prompt = agent._cached_system_prompt
    # ... pin session_start + session_id so any re-render path
    # still produces byte-identical output ...
    review_agent.session_start = agent.session_start
    review_agent.session_id = agent.session_id
```

The comment cites issue #25322 / PR #17276 and a measured **~26% end-to-end cost reduction on Sonnet 4.5** — a single-source, in-code figure, quoted as such. The pins are defensive in depth: the cached-prompt assignment already short-circuits the rebuild path; the session_start/session_id pins guarantee parity “even if a future code path bypasses the cache.” Routing review to a cheaper auxiliary model (auxiliary.background_review.{provider,model}) makes the parent’s cache useless — wrong cache key — so the routed fork instead replays a compact digest (last 24 messages verbatim plus a synthetic summary) rather than the full transcript (background_review.py:34–43, 122–163). The cloner’s lesson: cache parity is what makes per-ten-turn reflection *economically viable at all*, and it doubles as a constraint — learned artifacts live in the system prompt, so writes take effect only at session boundaries, which is precisely chapter 6’s frozen-snapshot behavior.

7.2.4 /learn, the learning graph, and closing the loop

The background cadence is opportunistic; /learn is the deliberate path. build_learn_prompt(user_request) builds one foreground instruction: gather named sources — directories, URLs, the current conversation, pasted notes — and author exactly one SKILL.md via skill_manage action=create (agent/learn_prompt.py:99–150). There is “no separate distillation engine and no model-tool footprint” (learn_prompt.py:18–22): distillation is the same loop with a different prompt. The embedded authoring standards (learn_prompt.py:30–96) enforce house style as hard rules — description ≤60 characters because the prompt index truncates at 60, author: Hermes literally for privacy, “NEVER

invent flags, paths, or APIs” — because a learned skill that lies about an interface is worse than none.

Visibility is a feature, not an afterthought. `agent/learning_graph.py` builds a presentation graph directly over on-disk state: skill nodes carry provenance, use counts, lifecycle state, and declared `related_skills` edges; `MEMORY.md`/`USER.md` split on `§` separators into memory “cards” linked to skills by lexical overlap (`learning_graph.py`:156–168, 227–245), filtered to learned artifacts with real signal (`learning_graph.py`:262–267). User-initiated mutations (`hermes journey delete|edit`, TUI, GUI) operate on the same node IDs — `delete` means *archive* for skills, atomic rewrite for memory — and every mutation clears the skills prompt cache so the next session reflects it (`agent/learning_mutations.py`:200–206).

The loop closes through the prompt, never through code: artifacts written at turn *N* sit on disk until the next session assembles its system prompt — skills index into the stable tier, memory snapshot into the volatile tier (chapters 5–6) — and only then alter behavior. Closure is real but **session-deferred**; the deferral is the cache invariant’s price, paid willingly.

7.3 Verification and the Curator

7.3.1 The verification ledger: the only in-loop verifier

The “no verifier” claim of §7.1.1 has one exception, and its scope deserves precision. `agent/verification_evidence.py` is a deliberately passive SQLite ledger — it “never decides to run a suite, never blocks completion” — classifying terminal-tool results into `lint/typecheck/build/test` events with canonical-command equivalence (`pytest` $\equiv$ `python -m pytest` $\equiv$ `uv run pytest`) and per-workspace edit tracking (`agent/verification_evidence.py`:1–6, 177–197). `agent/verification_stop.py` consumes it: when the model tries to end a turn after editing verifiable code — docs and prose filtered out, so `SKILL.md` edits never demand tests — without fresh passing evidence, it injects a synthetic user message demanding the verification command, bounded to two attempts per turn, default-on for CLI/TUI/desktop and default-off for messaging surfaces (`verification_stop.py`:24–72, 135–170, 245–310). Synthetic scaffolding is stripped from persisted history so it cannot poison resumed transcripts (`turn_finalizer.py`:50–66).

The scope discipline is the point. Verify-on-stop improves *truthfulness within a turn* — unverified completion claims become structurally difficult — and thereby improves the signal quality of the transcripts the fork later mines. It cannot measure whether a learned skill is good. Cross-turn artifact quality remains unverified by construction; the ledger is a floor, not a fitness function.

7.3.2 The Curator: deterministic decay, optional judgment

The Curator (`agent/curator.py`, 2,016 lines) is the lifecycle manager chapter 6 forward-pointed. Scheduling is inactivity-triggered, not cron: `should_run_now` gates on enabled-and-not-paused, `last_run_at` older than `interval_hours` (default **168 h**), an idle gate of **2 h** at the call site, and — unusually — a fresh install seeds `last_run_at = now`, so the **first run defers a full interval** rather than mutating a library it has never seen (`agent/curator.py`:233–283; cadence figures are Medium-tier: two readers, one source).

Phase 1 is a deterministic state machine with no LLM: `apply_automatic_transitions` walks agent-created skills by last real activity — unused 30 days — stale, 90 days — archive, used again while stale — reactivated (`curator.py:305–383`, predicate at `:370–381`). The guards encode operational respect: pinned skills skipped, cron-referenced skills skipped (a paused cron job’s skill is in use by definition), never-used skills granted a grace floor (“absence of evidence, not evidence of staleness”), hub-installed skills never pruned, and — the load-bearing invariant — **archive only, never auto-delete** (`curator.py:17`). Before every mutating pass the run takes a `tar.gz` snapshot of the skills tree plus `cron/jobs.json`, kept five-deep and undoable via `hermes curator rollback` — a rollback that is itself undoable (`agent/curator_backup.py:1–40`). The snapshot is best-effort by deliberate reasoning, at `curator.py:1544–1551`:

```
# Pre-mutation snapshot – best-effort, never blocks the run. A
# failed snapshot logs at debug and continues (the alternative is
# that a transient disk issue silently disables curator forever,
# which is worse). Users who want to require snapshots can disable
# curator entirely until they can fix disk space.
try:
    from agent import curator_backup
    snap = curator_backup.snapshot_skills(reason="pre-curator-run")
```

Phase 2 is the honest tell about LLM judgment here: an umbrella-building consolidation pass (“If you end the pass with fewer than 10 archives, you stopped too early”) that merges prefix clusters, demotes siblings to support files, and emits a structured YAML summary (`curator.py:417–568`) — and it is **off by default** (`DEFAULT_CONSOLIDATE = False`, `curator.py:78`). The maintainers treat aggressive LLM consolidation as risky enough to gate. Decay is deterministic; qualitative judgment is optional. That split is the Curator’s thesis.

7.4 Fitness Measurement Lives Elsewhere

7.4.1 The self-evolution companion: optimization as a PR, never a commit

The only fitness-measured optimization in the ecosystem is a separate repository, `NousResearch/hermes-agent-self-evolution`, which “operates ON hermes-agent — not part of it,” requires zero changes in the agent repo, and lands every result as a human-reviewed pull request.^{10 11} Its primary engine is DSPy with GEPA — reflective Genetic-Pareto prompt evolution, an ICLR 2026 Oral method that reads execution traces to learn *why* candidates fail¹² — with MIPROv2 as fallback; no GPU training, a reported ~\$2–10 per run in API calls.^{10 11} Phase 1, implemented as of June 2026, optimizes `SKILL.md` files against synthetic or session-mined eval sets with LLM-as-judge rubrics, behind constraint gates: full test suite green, skills ≤15 KB, cache compatibility (no mid-conversation prompt mutation), semantic preservation, benchmark gates, and a “done when” of ≥10% score increase with a diff that “reads sensibly to a human.”¹¹ Phases 2–5 (tool descriptions, prompt sections, tool *code*, continuous loop) are

¹⁰ Nous Research. `hermes-agent-self-evolution`: README + PLAN.md. <https://github.com/NousResearch/hermes-agent-self-evolution>

¹¹ The Agent Report. `Hermes Agent Self-Evolution`: Nous Research Ships Genetic Prompt Optimization with DSPy + GEPA. <https://the-agent-report.com/2026/06/hermes-agent-self-evolution-dspy-gepa-june2026/>

¹² Modern AI Guide. Programmatic prompt optimization (DSPy, GEPA). <https://modernaiguide.dev/docs/stack/prompt-optimization>

planned; the code phase carries the strictest rule — “every line of evolved code reviewed before merge.”¹⁰

The architectural relationship summarizes the chapter: the main repo’s loop is *experience* → *markdown*; the companion is *markdown* → *measurably better markdown* — “the existing Curator prunes unused skills; self-evolution *improves* the ones you keep.”¹¹ Together: create (background review, /learn) → maintain (Curator) → evolve (GEPA PRs). The project trusts autonomous improvement exactly up to the point where fitness becomes measurable, and not one step further: everything with an objective score passes through human review. [INFERRED] The split reads as a deliberate trust boundary rather than an accident of repo organization, though no source states the motivation.

7.4.2 The guardrail stack and residual risks

The chapter’s transferable content is the guardrail stack — each guard with its enforcement point and the failure it prevents:

Guard	Enforcement point	Failure mode prevented
Thread-scoped tool whitelist (memory/skill only)	agent/background_review.py:819–835	Reflection fork invoking shell, file, or network tools unsupervised
Persistence isolation (<code>_persist_disabled</code>)	<code>agent/background_review.py:743–754</code>	“Curator-takeover”: fork prompt injected into the real session, re-read as a standing instruction
Compression disabled; <code>_end_session_on_close=False</code>	<code>agent/background_review.py:790–807</code>	Fork winning a compression race or finalizing the parent’s live session (#38727)
Dangerous-command auto-deny	<code>agent/background_review.py:637–644</code>	Blocking <code>input()</code> deadlock against the parent TUI (#15216)
Provenance ContextVar + ownership guards	<code>tools/skill_manager_tool.py:296–395</code> ; <code>tools/skill_provenance.py</code>	Autonomous writes to pinned, bundled, hub-installed, or user-owned skills
Read-before-write preflight	<code>tools/skill_manager_tool.py:399–427</code>	Blind patches against unread or drifted skill content
Size caps (100 KB / 1 MiB)	<code>tools/skill_manager_tool.py:170–171, 488–489</code>	Unbounded artifact growth inflating every future system prompt
Archive-never-delete + snapshot + rollback	<code>agent/curator.py:17, 1544–1562</code> ; <code>agent/curator_backup.py</code>	Irrecoverable loss from a bad autonomous pass
Opt-in staging	<code>tools/</code>	Unreviewed or dangerous-

Guard	Enforcement point	Failure mode prevented
(write_approval) and scan (guard_agent_created)	skill_manager_tool.py:1279–1339, 94–141	pattern agent text entering the prompt — both off by default
Anti-capture prompt rules	agent/background_review.py:260–275	Negative tool claims hardening “into refusals the agent cites against itself for months”

The stack’s shape matters more than any row. Every always-on guard is a *capability* guard — whitelist, persistence isolation, archive-only, caps — enforced in code at the toolset layer (insight 4), while every *judgment* guard — content scanning, human approval, LLM consolidation — is opt-in. The default posture is therefore structurally safe and editorially permissive: the system cannot easily hurt itself, but nothing by default checks whether what it learned is *good*. Read-before-write is enforceable only because the reviewer is a full agent able to call `skill_view` — the practical argument for the heavyweight fork. The provenance flag is the keystone: it lets the Curator’s decay apply to agent sediment while never touching user property, and it must exist before the first autonomous write, because it cannot be retrofitted.

Residual risks, assessed neutrally:

- **Skill-quality drift (medium).** Bias-to-act prompting with no fitness signal accumulates plausible-but-wrong procedures; the Curator prunes by *use*, not *correctness*. A weak review model degrades output gracefully — noise, not corruption — but noise compounds in a prompt loaded every session.
- **Ledger as sole verifier (medium).** Verify-on-stop covers code edits only; learned Markdown bypasses it by design. A systematically wrong skill faces no in-loop challenge until a human notices or the external pipeline scores it.
- **Default-off judgment gates (medium in untrusted environments).** `guard_agent_created` and `write_approval` are opt-in; deployments ingesting adversarial content should flip both and accept the review burden.
- **Prompt-mediated learning ceiling (low, inherent).** The loop can only learn what transcript review can see; it cannot discover improvements requiring measurement. A scope limit, not a defect — but do not expect the in-repo loop to move task success rates measurably.
- **Cache-parity fragility (low).** The pins couple the fork to prompt-assembly internals; a future change that perturbs the prefix silently forfeits the ~26% saving — the code’s own defensive comments anticipate this.

7.4.3 Clone notes

Clone notes. The learning fork is stage 8 of the staged roadmap (~400 LOC: cadence counters, a whitelisted second-agent fork with `_persist_disabled`, curator-lite stale/archive transitions with `snapshot/rollback`) and the first omission candidate — the loop runs full turns without it, and a time-pressed clone should start with `/learn-style` foreground distillation only. If the fork is kept, the guardrail stack is non-negotiable: whitelist, persistence isolation, provenance, archive-never-delete, and

snapshot/rollback are the difference between “self-improvement” and an unsupervised agent writing its own future prompts. Skip the cache-parity pins if the target provider lacks prefix caching (the routed-model digest path is the honest fallback), and flip `write_approval` and `guard_agent_created` on by default if the clone ingests untrusted content. Budget honestly: without cache parity, reflection roughly doubles token spend on review turns [INFERRED from the ~26% figure cited at `background_review.py:772–774`] — the cadence counters, not the fork, are where the cost knob lives.

8. Delegation and Proactivity: Subagents, `execute_code`, and Cron

Hermes gives the agent three ways to make work happen outside the current turn: `delegate_task` fans reasoning-heavy work out to subagents, `execute_code` collapses mechanical tool chains into one scripted call, and `cronjob` schedules the agent’s own future runs. The surfaces differ in isolation, delivery, and cost, but share one governing rule, stated once here as this chapter’s spine (cross-cutting insight 4): **capability subtraction beats instruction**. Every recursion or runaway risk these surfaces create is mitigated by removing tools from the spawned context’s schema in code — a tool absent from the schema is one the model physically cannot call — never by instructing the model to refrain. Prompt-layer hints (the cron schema’s “should not recursively schedule more cron jobs,” `tools/cronjob_tools.py:988`) are second-tier reinforcement over a hard control, not the control itself.

8.1 Subagent Delegation

8.1.1 In-process thread-based subagents

A `delegate_task` call builds a brand-new `AI Agent` per task on the calling thread — fresh messages list, ephemeral system prompt assembled from goal + context, no memory or context files — and runs it on daemon `ThreadPoolExecutor` workers inside the same Python process (`tools/delegate_tool.py:1366–1407`; `agent/delegation_context.py:4–6`). Fan-in is summaries only: the parent’s context sees the delegation call and a bounded summary string, never the child’s intermediate tool calls; over-budget summaries are head/tail-trimmed with the full text spilled to disk (`tools/delegate_tool.py:1644–1791`).

The in-process thread model buys cheap spawns, a shared credential pool, and zero serialization of parent state; it forfeits hard isolation. Python cannot kill a wedged thread (interrupts are cooperative flags), and only convention plus per-`task_id` namespacing of terminal environment and `cwd` stops a child from mutating shared process state. Process isolation would add hard kills and memory separation at the price of pickling state, slower spawn, and duplicated provider clients. Hermes accepts threads because the isolation it needs is *conversational* — separate message lists, summarized fan-in — not memory safety.

The model does not choose sync versus async. The dispatch intercept (`run_agent.py:6493–6523`) forces background for top-level agents — the schema’s `background` parameter is ignored — and synchronous execution for orchestrator children, which need worker results inside their own turn and do not own the gateway session an async result would route back to. The trade-off is forced-background delegation versus interactive latency: the parent never blocks, but

results re-enter as later turns, so latency-sensitive composition must go through the orchestrator role. Sessions that cannot receive a detached result (one-shot hermes -z, cron runs, stateless HTTP) degrade to synchronous ([tools/delegate_tool.py:2925–2974](#)).

8.1.2 Hard guards: the blocklist and the depth cap

The primary guard is a constant, quoted from [tools/delegate_tool.py:46–54](#):

```
DELEGATE_BLOCKED_TOOLS = frozenset([
    "delegate_task", # no recursive delegation
    "clarify", # no user interaction
    "memory", # no writes to shared MEMORY.md
    "send_message", # no cross-platform side effects
    "cronjob", # no scheduling more work in the parent's name
])
```

Two further layers sit behind it: a toolset strip intersected with the parent's own toolsets, so a child cannot gain tools the parent lacks, and exact one-tool deny toolsets that survive composite bundles such as hermes-cli ([tools/delegate_tool.py:766–805](#)). There is no model-facing toolsets argument; the sanctioned exception is `role="orchestrator"`, which re-adds the delegation toolset only when the kill switch allows and depth permits.

Risk callout — unbounded depth when raised (severity: HIGH).

`delegation.max_spawn_depth` defaults to 1 (flat tree) with a floor of 1 and *no ceiling* ([tools/delegate_tool.py:467–503](#); entry guard at :2481–2494). Raising it removes the only depth bound in the system: leaves grow as $\text{max_concurrent_children}^{\text{depth}}$ — the in-repo documentation notes a $3 \times 3 \times 3$ tree reaches 27 concurrent leaves. Existing mitigations cap other axes (`max_concurrent_children` rejects rather than queues; a spawn-pause kill switch; per-child iteration budgets), but none caps depth; operators raising this knob should treat it as an explicit cost multiplier.

8.1.3 Durable completion and idle-only re-entry

Async results return as a completion event on the shared `process_registry.completion_queue`, forged into a brand-new turn. Per the module header ([tools/async_delegation.py:15–22](#)), completions surface as a new turn only when the agent is idle, never spliced between a tool result and an assistant message — the hard invariant “never mutate past context.” This is the role-alternation invariant cross-referenced from chapter 5's cache economics: mid-turn injection would corrupt both the message contract and the byte-stable prompt prefix.

Delivery is durable, not best-effort. Completions persist to a `state.db` table behind a claim/ack protocol — 300 s claim lease, complete/release/drop transitions, `_MAX_DELIVERY_ATTEMPTS = 8` before a terminal dropped state ([tools/async_delegation.py:323–424](#)). On restart, recovery marks children of provably dead owner PIDs unknown and re-enqueues undelivered completions: at-least-once with bounded retries.

All of this runs on the shared `DaemonThreadPoolExecutor` ([tools/daemon_pool.py:1–64](#), 64 lines), whose daemon workers escape the `stdlib atexit` join that once caused multi-minute CLI exits. Four subsystems reuse it — async delegation, the tool executor (chapter 3’s concurrency layer), the memory manager, and the skills hub. The reuse is an operability decision: one exit-safety fix propagates everywhere, but it couples those subsystems’ lifecycle behavior, and the async executor grows to meet demand yet never shrinks ([tools/async_delegation.py:464–472](#)).

8.2 Programmatic Tool Calling

8.2.1 *execute_code*: RPC back into the real dispatcher

`execute_code` is the second surface and the opposite trade: instead of spawning an agent, it spawns a script. The LLM writes Python; a generated `hermes_tools.py` stub module serializes each allowed call over a token-authenticated `AF_UNIX` socket (file-based RPC on remote backends) into *the same dispatcher the agent loop uses* ([tools/code_execution_tool.py:618–670](#)). The script runs in a subprocess with a scrubbed environment (secret-substring blocklist plus safe-prefix allowlist), deferred to chapter 9’s treatment of execution environments.

The boundary conditions are the interesting part (figures are Medium tier, attributed by method: single code read of the module’s constants, dimension 07). The stub generator admits a hard 7-tool allowlist, `SANDBOX_ALLOWED_TOOLS = {web_search, web_extract, read_file, write_file, search_files, patch, terminal}`, intersected with session-enabled tools and enforced again server-side ([tools/code_execution_tool.py:62–70](#)). Caps are `DEFAULT_MAX_TOOL_CALLS = 50` and `MAX_STDOUT_BYTES = 50_000` with head/tail truncation ([tools/code_execution_tool.py:72–76](#)). Only the capped stdout re-enters the model context, so loops, filtering, and pagination cost zero intermediate tokens.

Since the script is arbitrary Python that never passes through the terminal tool’s dangerous-pattern checks, the whole script is gated before spawn, from [tools/code_execution_tool.py:1223–1230](#):

```
# execute_code runs arbitrary Python (subprocess/os.system/...) that never
# passes through terminal()/DANGEROUS_PATTERNS, so guard the whole script
# here before either dispatch path spawns it. Runs synchronously in the
# caller (tool-executor) thread, which holds the session context (#30882).
# A Docker sandbox with host bind mounts is no longer isolated, so its
# script does not get the container fast-path.
from tools.approval import check_execute_code_guard
_guard = check_execute_code_guard(
```

The approval is whole-script and pre-execution: one decision covers every tool call the script will make, the only workable granularity when the call sequence is data-dependent. Honest residue, carried verbatim from the evidence base: [INFERRED] Residual risk: the script runs with the user’s local UID (strict mode) or project venv (project mode) — it is *not* a sandbox in the VM sense unless the terminal backend is remote; the guardrails are allowlist+approval+env-scrub, not seccomp.

8.3 Cron: Proactivity with Hard Isolation

8.3.1 A fresh isolated session per fire

Cron is the third surface: the agent schedules its own future execution. Each fire builds a fresh, isolated AIAgent session — id `cron_{job_id}_{timestamp}`, no memory, no conversation history, session contextvars cleared so the run never impersonates a live user turn ([cron/scheduler.py:2966–3030](#)). The recursion guard is this chapter’s second canonical capability-subtraction instance, from [cron/scheduler.py:156–176](#):

```
def _resolve_cron_disabled_toolsets(cfg: dict) -> list[str]:
    """Toolsets a cron-spawned agent must never receive.
    Three protected toolsets are always disabled in cron context:
    - ``cronjob`` – would let a cron-spawned agent schedule more cron
    jobs
    - ``messaging`` – interactive, needs a live gateway session
    - ``clarify`` – interactive, blocks waiting for user input
    ...
    disabled = ["cronjob", "messaging", "clarify"]
```

The strip is layered with the user’s config denylist so per-job enabled_toolsets cannot widen past policy, and is passed to AIAgent(disabled_toolsets=...) at [cron/scheduler.py:3417](#). A second hard layer reinforces it: the cronjob tool’s registration check_fn requires interactive environment flags that cron sessions never set, so the tool never enters the schema at all.

8.3.2 The LLM as schedule parser — an unusual bet, interrogated

There is no natural-language date parser anywhere in the cron layer. The user says “every morning at 9am”; the model emits a structured schedule argument, and parse_schedule accepts exactly four shapes — “30m” (once), “every 30m” (interval), a croniter-validated cron expression, or an ISO timestamp ([cron/jobs.py:512–609](#)). The bet: schedule translation is a language task for the model already in the loop, not a parsing problem owned by the scheduler. It pays three ways — no parser dependency to maintain, colloquial recurrence phrasing handled for free, and a small closed output space that is cheap to validate. It costs a silent failure mode: a mistranslated schedule is wrong until the first missed or extra fire, and nothing can detect a semantically wrong-but-well-formed expression. The mitigations are structural, not algorithmic: past one-shots are rejected at create time, and the four-shape whitelist bounds the blast radius. This is the one place in the cron layer where correctness rests on the model rather than on code.

8.3.3 At-most-once, by design

Each tick takes a cross-process file lock, finds due jobs, and advances next_run_at for all of them *before any execution begins* ([cron/scheduler.py:4029–4035](#)); finite one-shots additionally consume a durable claim_dispatch before their side effect runs. The consequence is at-most-once: a crash after the advance but before execution is a *missed* run, never a duplicate. Weighed against exactly-once, this is defensible — exactly-once delivery to chat channels would require a durable intent log plus idempotent side effects at every target, a distributed-transaction cost

that buys little for scheduled summaries. The executions ledger is an audit state machine, not a recovery mechanism; its docstring says so verbatim ([cron/executions.py:1–6](#)): “The ledger records what is known about each attempt; it is not a retry queue.” Interrupted attempts become unknown only after their owner process is proved gone; a failed recurring job simply fires at its next occurrence.

Proactivity enters only through consent. Suggestions never auto-create jobs; acceptance calls the same `create_job`, dismissals latch on a stable `dedup_key`, and the backlog is capped at five ([cron/suggestions.py:18–22](#)). Blueprints parameterize only human-friendly slots (time-of-day, weekday set, interval minutes) over fixed recurrence templates, and `fill_blueprint` returns `create_job` kwargs: one schema, no second job engine ([cron/blueprint_catalog.py:661–674](#)).

8.3.4 The three surfaces compared

Surface	Isolation level	Capability restriction mechanism	Completion / delivery semantics	Guard type
Subagent (<code>delegate_task</code>)	Thread in same process; fresh AI Agent + fresh message list; per-task_id env namespacing	DELEGATE_BLOCKED_TOOLS frozenset + toolset strip $\cap$ parent + depth guard + kill switch	Bounded summary string (sync) or idle-only new turn (async); durable claim/ack, 8 attempts then dropped	Hard (code-enforced)
<code>execute_code</code> (RPC)	Subprocess with scrubbed env; token-authenticated AF_UNIX RPC into the real dispatcher	7-tool allowlist enforced server-side + whole-script pre-execution approval	Only capped stdout (≤ 50 KB) re-enters context as the tool result	Hard (allowlist + approval)
Cron job	Fresh isolated AI Agent session per fire; no history; session contextvars cleared	<code>cronjob/</code> messaging/clarify toolsets stripped in code; <code>check_fn</code> registration gating	Final response auto-delivered to channels at fire time; at-most-once via advance-before-execute	Hard, plus second-tier prompt hint

The right-hand column is uniform, and that uniformity is the chapter’s argument: across three surfaces with different threat models, the primary guard is always capability removal in code, with prompt-layer text only as backup. The isolation column forms a gradient — thread, then subprocess, then fresh session — but it does not track risk naively: cron, which runs the *fullest* agent of the three, compensates with the strongest pre-spawn strip and the most conservative delivery semantics, while the least-isolated subagent gets the most engineered return path precisely because it couples tightly to the parent’s context. Delivery semantics invert with

coupling: the closer a surface sits to the parent conversation, the more machinery surrounds the result's re-entry; the further away, the more delivery leaves the context entirely. Note also what no surface offers: exactly-once. The system prefers auditable misses and bounded retries over duplicate side effects, and encodes that preference in constants and docstrings rather than in policy prose.

8.3.5 Clone notes

One related surface deserves a paragraph first. `batch_runner.py` is the datagen-facing cousin of `delegate_task`: it parallelizes across *processes* (`multiprocessing.Pool`, not threads), builds a fresh `AIAgent` per prompt with per-`task_id` environment overrides, and resumes interrupted runs by content rather than index. Its coupling points are treated in chapter 9's research-seams section; here it matters only as evidence that the thread-based default was a choice.

Clone notes. In the staged build order, delegation is stage 9 and cron is stage 10 — both omissible from a minimal core, which runs complete turns with provider path, loop, registry, environment, and approval alone. If adopted, delegation is roughly 270 lines: a child factory (fresh agent, goal+context prompt, parent toolsets minus blocklist), a pool runner using `wait(FIRST_COMPLETED, timeout=0.5)` so interrupts stay responsive, a summary gate with head/tail trim and disk spill, and a depth attribute with an entry check. Implement the hard blocklist on day one — about 50 lines prevents the entire runaway-recursion class; the durable claim/ack layer can wait unless the deployment is multi-process. Cron is roughly 300 lines: a JSON job store with atomic writes, a tick loop, a fresh-session executor with the toolset strip, and advance-before-execute. Defer `execute_code` RPC scripting — a second dispatch surface with its own authentication, presupposing a stable dispatcher. Invariants to preserve in either case: remove capabilities in code rather than instructing around them; advance or claim before side effects; re-enter async completions only between turns; give every spawned context a fresh message list with summaries-only fan-in.

9. The Platform Layer: Environments, Security, Providers, and Research Seams

The preceding chapters treated the agent as a loop plus the data the loop reads. This chapter descends one level, to the platform the loop stands on: the execution environments that run its shell commands, the security model that decides which commands may run, the provider stack that decides which model answers, and — as the closing case study — the research seams that once connected this runtime to an RL training stack and were severed in a single pull request without destabilizing anything. The through-line is insight 6: the parts of this platform that survived contact with removal are the ones built as narrow, named seams.

9.1 The Trust Model, Quoted and Taken Seriously

9.1.1 One boundary, and it is not in the process

Hermes Agent's security policy makes a claim most agent frameworks avoid making about themselves, and it makes it in writing. From `SECURITY.md`:60-65 ([permalink](#)):

The only security boundary against an adversarial LLM is the operating system. Nothing inside the agent process constitutes containment — not the approval gate, not output redaction, not any pattern scanner, not any tool allowlist. Any in-process component that screens LLM output is a heuristic operating on an attacker-influenced string, and this policy treats it as such.

The architectural consequence is total delegation: containment lives in the environment layer (section 9.2) or in whole-process wrapping chosen by the operator, and everything in-process — the approval gate, the redactor, the danger-pattern scanner — is classified as a heuristic over attacker-influenced strings. This is not the policy conceding defeat; it is the policy assigning each component an honest role. The in-process layer is built to catch cooperative-mode mistakes and to raise the cost of injection, and the code says so itself: the denylist “is structurally incomplete... catches cooperative-mode mistakes, not adversarial output” (SECURITY.md:142–146), and environment scrubbing “reduces casual exfiltration. It is not containment” (SECURITY.md:121–131). What real-world exploit resistance this buys is out of scope here; what is verified is that the code’s defaults behave the way the policy says they do.

9.1.2 The approval gate: chain-of-responsibility with a floor below yolo

`check_dangerous_command` (tools/approval.py:2877–2944, [permalink](#)) evaluates a command through an ordered chain: (1) container-guard skip — isolated backends bypass the gate entirely, but local and ssh *never* skip (approval.py:2863–2874); (2) a hardline floor — `rm -rf /`, `mkfs`, `dd-to-device`, `shutdown`, `fork bombs` — blocked unconditionally, evaluated *before* yolo mode so no operator flag can lower it (approval.py:2898–2906); (3) user deny rules, also before yolo; (4) the yolo bypass; (5) a persistent shell-operator-aware allowlist; (6) multi-variant deobfuscated pattern detection; (7) the interactive/smart/gateway decision core. The defaults fail closed and that is verified behavior, not aspiration: an approval timeout denies (default 300 s), an absent human denies, cron sessions deny unless explicitly configured otherwise, and an unscoped secret read under profile multiplexing raises `UnscopedSecretError` rather than risk reading another profile’s key (agent/secret_scope.py:123–177).

The gate’s most instructive detail is a four-line constant at import time, tools/approval.py:32–35 ([permalink](#)):

```
# Freeze YOLO mode at module import time. Reading os.environ on every call
# would allow any skill running inside the process to set this variable
and
# instantly bypass all approval checks – a prompt-injection escalation
path.
_YOLO_MODE_FROZEN: bool = is_truthy_value(os.getenv("HERMES_YOLO_MODE",
""))
```

The same treatment protects redaction: `_REDACT_ENABLED` is snapshotted at import so an LLM-issued `export HERMES_REDACT_SECRETS=false` cannot disable it mid-session (agent/redact.py:61–69). This is the third canonical instance of the report’s capability-subtraction pattern (insight 4): alongside the delegation blocklist and the cron toolset strip, the system defends itself not by instructing the model to behave but by removing the lever the

model could pull. Session identity is likewise held in a contextvar, not an environment variable, because concurrent executor threads once raced on `os.environ` and dropped a session onto the non-interactive auto-approve path (GHSA-96vc-wcxf-jjff; `approval.py:54-66`).

9.2 Six Backends, Two Methods

9.2.1 A two-method contract carrying the whole system

Every terminal backend in Hermes is a `BaseEnvironment` subclass, and the contract each must honor is two methods. `tools/environments/base.py:390-396` ([permalink](#)):

```
class BaseEnvironment(ABC):
    """Common interface and unified execution flow for all Hermes
    backends.

    Subclasses implement ``_run_bash()`` and ``cleanup()``. The base
    class
    provides ``execute()`` with session snapshot sourcing, CWD tracking,
    interrupt handling, and timeout enforcement.
    """
```

Everything else — the spawn-per-call `bash -c` flow, the atomically written session snapshot that lets `cd` and exported variables survive across separate processes, the interrupt/timeout/drain loop, bounded output capture — is inherited template method (`base.py:1045-1105`). File tools have no backends of their own: `ShellFileOperations` is a facade over the same shell contract, “implemented on top of the shell contract — they cannot reach paths the backend doesn’t expose” (`tools/file_operations.py:793-799`; the security consequence is stated at `SECURITY.md:72-76`). The result is that a command’s containment properties are entirely a function of which backend the factory (`tools/terminal_tool.py:1484-1633`, keyed off `TERMINAL_ENV`) instantiates:

the six terminal backends and their trust/containment characteristics (`tools/environments/;` `docker hardening at docker.py:336-344`; `approval skip rules at approval.py:2863-2874`).

Backend	Execution mechanism	Containment boundary	Approval gate	Persistence model
local	host <code>bash -c</code> , own process group	none — host trust envelope	always on	host filesystem
docker	docker exec into long-lived container	the container (cap-drop ALL, no-new-privileges, optional <code>--network=none</code>)	skipped only when no host paths bind-mounted	bind mounts or tmpfs per task
ssh	ssh ... <code>bash -c</code> over <code>ControlMaster</code>	network boundary only — someone	always on	remote FS + two-way .hermes sync

Backend	Execution mechanism	Containment boundary	Approval gate	Persistence model
		else's machine		
singularity	apptainer exec instance://...	--containall --no-home container	always skipped	per-task overlay or writable tmpfs
modal (direct/managed)	sandbox.exec / gateway HTTPS	cloud VM sandbox	always skipped	filesystem snapshot → restore as image
daytona	sandbox.process.exec	cloud VM sandbox	always skipped	stop/resume of named sandbox

Two properties of this table deserve emphasis. First, the asymmetry in the approval column is deliberate and load-bearing: the gate exists to protect the *host*, so it is skipped exactly where an OS-level boundary already stands between the command and the operator's machine — and docker only earns the skip when no host path is bind-mounted into the container, a check (`_docker_has_host_access`) that keeps the convenience of bind-mounted workspaces from silently disabling scrutiny. Second, containment quality is purchased per backend, not per system: local offers none by design, ssh offers someone else's machine, and the cloud backends offer a VM the agent can burn. An operator choosing `TERMINAL_ENV` is choosing the security posture; the code makes no attempt to blur that choice. The concentration of trust is the corresponding risk: because the contract is two methods, every containment guarantee reduces to the correctness of one backend's `_run_bash()` and its platform configuration, and the policy in 9.1.1 is what keeps that concentration honest rather than hidden.

9.3 The Five-Layer Provider Architecture

9.3.1 *api_mode* as the central discriminator

Provider support is split across five cooperating layers, and the separation is the single most important thing to understand before cloning:

the five provider layers with responsibility and plugin surface. The count of 33 bundled profile plugins is a directory-listing count of `plugins/model-providers/` from a single dimension pass (Medium tier; deterministic method, one counter).

Layer	Location	Responsibility	Plugin surface
Auth registry	<code>hermes_cli/auth.py</code> (<code>PROVIDER_REGISTRY</code>)	provider identity, auth type, env-var priority	static dataclass entries
Declarative profiles	<code>providers/base.py</code> , <code>plugins/model-providers/</code>	per-provider behavior: <code>base_url</code> , headers, quirks via hooks	33 bundled plugins, lazily discovered
Runtime resolver	<code>hermes_cli/</code>	(provider, config,	<code>models.dev</code> overlay +

Layer	Location	Responsibility	Plugin surface
	<code>runtime_provider.py</code> , <code>hermes_cli/providers</code> <code>.py</code>	env, auth store) → concrete client parameters	user config
Transports	<code>agent/transport/</code>	OpenAI-shaped state → provider wire format, per <code>api_mode</code>	<code>register_transport()</code> registry
Native adapters	<code>agent/</code> <code>anthropic_adapter.py</code> et al.	client construction, token lifecycle, message/tool translation	per-mode adapter modules

The design’s keystone is that only two of these layers touch the wire, and they are keyed by one string: `api_mode` — `chat_completions` (default), `anthropic_messages`, `codex_responses`, `bedrock_converse`, or `codex_app_server` (`agent/agent_init.py`:581-612). The internal representation is OpenAI-shaped end to end — messages, tools, response objects — and each transport’s job is confined to converting that invariant shape to and from one provider’s wire format (`build_kwargs`, `normalize_response`; `agent/transport/base.py`:16-60). The profile layer is where provider quirks live declaratively: hooks such as `prepare_messages`, `build_extra_body`, and `get_max_tokens` replaced what the module docstring describes as “20+ boolean flags” (`providers/base.py`:1-10), and profiles explicitly “do NOT own client construction, credential rotation, or streaming.” Trust in this ring is fail-closed in insight 5’s sense: plugin LLM access requires per-plugin opt-in flags before a plugin may override provider or model, and unscoped plugins never see credentials (`agent/plugin_llm.py`). The drift-prone surface is the reasoning-translation shims — reasoning-mandatory models that reject any reasoning field, signed thinking blocks that must replay verbatim and in order, encrypted reasoning stamped per issuer — each a 400-error mine encoded in adapter code.

9.3.2 Resolution priority: explicit intent wins, OAuth is the last resort

`resolve_provider()` (`hermes_cli/auth.py`:1847, chain documented at `auth.py`:1857-1869, [permalink](#)) resolves “which provider?” through an eight-step chain: explicit CLI key → `config.yaml` → OpenRouter env vars → credential pool → provider-specific keys → `auth.json` OAuth login → AWS credential chain → error. The ordering decision that matters operably is step 6: a logged-in OAuth provider is *demoted to last resort* (issue #29285), because explicit user intent should win over a stale login — a stale OAuth entry warns rather than silently hijacking the session. API keys are additionally scoped to hosts so an aggregator key is never sent to a custom endpoint (`hermes_cli/runtime_provider.py`:1132-1156). Alongside the main lane runs the auxiliary client lane (`agent/auxiliary_client.py`), which serves side tasks — titles, compression, vision, plugin LLM calls — with its own provider chain, per-task config, and `ContextVar`-based cost accounting. Resilience across both lanes is driven by the ~25-member `FailoverReason` taxonomy (`agent/error_classifier.py`:24-72), which classifies each failure into `retry` / `compress` / `rotate-credential` / `fallback` actions; the loop-level consumption of that seam is chapter 3’s subject and is not re-derived here.

9.4 Research Seams and the Amputation

9.4.1 The layer that is no longer there

Stale-source note (C2). External articles and the still-live documentation page (`hermes-agent.nousresearch.com/docs/developer-guide/environments`) describe an `environments/` directory — `HermesAgentBaseEnv`, `HermesAgentLoop`, `ToolContext`, tool-call parsers, a two-phase GRPO pipeline. Those sources are stale. The directory is absent at HEAD, and this is the one claim in this study verified against live history rather than the snapshot: the GitHub commits API for path `environments/hermes_base_env.py` returns removal commit `5af672c753`, dated 2026-05-15, PR #26106, “chore: remove Atropos RL environments and tinker-atropos integration”; the path 404s both at clone HEAD `4c9628e` and on live main.

Historically, the removed layer was a gym-style stack: an `Atropos BaseEnv` subclass that set `TERMINAL_ENV`, resolved Hermes tool schemas, and ran rollouts through an agent loop mirroring `run_agent.py`, with reward functions reaching back into the rollout’s own sandbox via a task-scoped `ToolContext` to verify real filesystem state. That is the full historical treatment this chapter gives it — what matters architecturally is not what was deleted but what the deletion did not touch: the loop, the tool dispatcher, and the environments layer all survived unchanged, because the coupling had been built as seams.

9.4.2 The seams that survived

surviving research seams, with location and what each decouples. The first three are the integration points the RL layer plugged into; the last two are durable assets that serve production independent of any training stack.

Seam	Location	What it decouples
<code>register_task_env_overrides</code>	<code>tools/terminal_tool.py:1125–1142</code>	per-rollout sandbox config (image, cwd) from environment construction
<code>per-task_id</code> sandbox isolation	<code>tools/terminal_tool.py:1191;</code> <code>batch_runner.py:347</code>	concurrent rollouts/tasks from shared container state
<code>_run_async</code> bridge	<code>model_tools.py:97–122</code>	sync tool handlers from foreign event loops (gateway, Atropos)
ShareGPT trajectory writer	<code>agent/trajectory.py:30–53</code>	training-data production from runtime control flow
three-layer tool-result budgeting	<code>tools/</code> <code>tool_result_storage.py:1–23</code>	context-window safety from any single tool’s output

The first seam is the most explicit — its docstring still names the consumer that no longer exists, `tools/terminal_tool.py:1125–1132` ([permalink](#)):

```
def register_task_env_overrides(task_id: str, overrides: Dict[str, Any]):  
    """  
    Register environment overrides for a specific task/rollout.
```

Called by Atropos environments before the agent loop to configure per-task sandbox settings (e.g., a custom Dockerfile for the Modal image).

```
... # docstring continues: supported override keys and args
"""
```

Each row earns its place differently. The override registry and `task_id` isolation let an external driver configure and fence a sandbox per rollout without the runtime knowing what a rollout is; both now serve the batch runner, ACP sessions, and multi-task isolation generally. The `_run_async` bridge keeps sync tool handlers callable from inside someone else's event loop by running the coroutine on a disposable worker thread — written for Atropos, still required by the gateway. The trajectory writer and budgeting layers are production assets that were always dual-use: budgeting keeps any model's context window safe from unbounded tool output, and the ShareGPT writer turns any session into an inspectable artifact. Nothing in the table required modification when PR #26106 landed; that is the evidence that the seams were real.

9.4.3 The lesson, with confidence marked

The architectural lesson is insight 6, now with its proof on the table: design training/eval coupling as injectable seams — narrow, named, callable from either side — so a research layer can be added *or removed* without forking the runtime. Hermes's research-readiness turned out to be seams, not subsystems, and the durable assets are precisely the ones that serve production too. [INFERRED] Nous moved the RL stack out of the OSS repo (to Atropos/Tinker-side or private); the agent repo kept only the generic datagen machinery. The removal itself is verified against the GitHub API; the motive and its impact remain [INFERRED] and are asserted as nothing more.

9.4.4 Clone notes

For the builder cloning the agent core: the provider path is stage 1 of the build order — a single `chat_completions` transport with the OpenAI-shaped invariant proves the architecture before any plugin machinery is earned — and the environment-plus-approval pair is stage 4: one `BaseEnvironment` (local) honoring the two-method contract, the file-tools facade over it, and a fail-closed approval gate with the hardline floor and the frozen yolo toggle, roughly 500 lines by the corpus's convergent estimate. The seam-design lesson of 9.4.3 is carried forward into chapter 10: the tool-result budgeting seam reappears there as a named pattern (entry 10) with its own build guidance, and the surviving seams generally anchor Part B's core-interface expectations and re-verify list.

10. Patterns Catalog, Coverage Map, and Clone Roadmap

The preceding chapters dissected mechanisms; this closing chapter abstracts them. Part A restates the mechanisms of Chapters 3–9 as handbook-liftable pattern entries, maps them onto the modules that instantiate them, and names the canonical patterns the system deliberately declines. Part B assembles the per-chapter “Clone notes” callouts into a staged build roadmap for the agent core. All code citations are pinned to this study's snapshot (HEAD 4c9628e, July 2026).

10.1 Catalog Admission Rules and Taxonomy Frame

10.1.1 Admission rule and reference frames

A pattern enters the catalog only if instantiated in code at the snapshot, however small the instance; patterns claimed in documentation or expected from the literature but absent from the code go to the ABSENT section (§10.3.2), never into entries. Entries are graded CANONICAL — a known pattern from the reference frames, instantiated here — or PIONEER, where no clean precedent exists and novelty is argued from the taxonomy evidence, not asserted.

Four frames supply the vocabulary: Anthropic’s workflow/agent split and five workflow patterns^{13 14}; Ng’s four cognitive patterns — Reflection, Tool Use, Planning, Multi-Agent¹⁵; the CoALA memory taxonomy with context engineering as curation of the working tier^{8 9 16}; and the long-running-agent literature — ReAct,^{17 1} approval tiers,^{18 19} heartbeat/cron proactivity,^{20 7} and the SKILL.md standard.^{21 22} Entries cite these under Related patterns; Chapters 3–9 remain the evidence base.

10.2 The Entries

Template, fixed order: Name (with literature aliases); Status; Intent; Problem/Context; Solution; Structure; Code evidence; Trade-offs; Related patterns. Each entry back-references its host chapter, which carries the full mechanism and figures.

10.2.1 Loop and tool-system entries (1–6)

1. Loop-as-State-Machine (ReAct loop) - **Status:** CANONICAL. - **Intent:** one while-loop per turn; control state in data, not graph topology. - **Problem/Context:** graphs scatter control flow; unbounded loops convert churn into spend.^{17 6} - **Solution:** run_conversation iterates build → call → dispatch → append until a tool-call-free response; state is _turn_exit_reason plus TurnRetryState’s one-shot restart flags. - **Structure:** Figure 2 (Chapter 3). - **Code evidence:** agent/conversation_loop.py:669–6166 ([permalink](#)); agent/turn_retry_state.py:32–79. - **Trade-offs:** ~3,900 lines; a recovery branch missing its one-shot guard loops forever. - **Related patterns:** entries 2, 3; ReAct^{17 1}.

2. Prologue/Epilogue Turn Seams - **Status:** CANONICAL. - **Intent:** per-turn setup/teardown in two replaceable seams outside the loop. - **Problem/Context:** policy interleaved with control flow forces every change into the hot path. - **Solution:** build_turn_context opens each turn

¹³ SCHLUNTZ E, ZHANG B. Building effective agents. <https://www.anthropic.com/research/building-effective-agents>

¹⁴ A Two-Dimensional Framework for AI Agent Design Patterns: Cognitive Function × Execution Topology. <https://arxiv.org/html/2605.13850v2>

¹⁵ Agentic Design Patterns: A System-Theoretic Framework. <https://arxiv.org/html/2601.19752v1>

¹⁶ Anthropic. Effective context engineering for AI agents. <https://github.com/jcosta33/corpus-skills/blob/main/docs/sources.md>

¹⁷ YAO S, et al. ReAct: Synergizing Reasoning and Acting in Language Models. <https://arxiv.org/pdf/2210.03629>

¹⁸ CreateOS. Human-in-the-Loop AI Agents: When Approval Gates Matter. <https://createos.sh/blogs/human-in-the-loop-ai-agents>

¹⁹ BetterClaw. AI Agent Guardrails: Human Approval Without Killing Speed. <https://www.betterclaw.io/blog/ai-agent-human-approval-guardrails>

²⁰ MindStudio. What Is the Agentic OS Heartbeat Pattern?. <https://www.mindstudio.ai/blog/agentic-os-heartbeat-pattern-proactive-ai-agent>

²¹ Agent Skills open standard. <https://agentskills.io/home>

²² Harness Engineering for Agentic AI Coding Tools: An Exploratory Study. <https://arxiv.org/html/2602.14690v5>

(sanitization, prompt restore-or-build, session row, preflight compression); `finalize_turn` closes every exit (budget-summary fallback, persistence); both extracted behavior-neutrally. - **Structure:** Figure 2 (Chapter 3). - **Code evidence:** `agent/turn_context.py`, invoked at `agent/conversation_loop.py:729–748`; `agent/turn_finalizer.py:69` ([permalink](#)). - **Trade-offs:** prologue cost is paid every turn; a seam bug degrades all turns uniformly. - **Related patterns:** entries 1, 3, 9.

3. Iteration Budget with Consume/Refund - **Status:** CANONICAL. - **Intent:** bound per-turn spend without charging recovery restarts against progress. - **Problem/Context:** AutoGPT-class runaway is the canonical autonomy failure ⁶; flat caps punish legitimate recovery. - **Solution:** thread-safe `consume()` gates each iteration (default 90; 50 subagents); targeted `refund()` for compression/redirect/rebuild restarts; exhaustion degrades to one tool-less summary call. - **Structure:** loop-top gate plus finalizer fallback (Chapter 3). - **Code evidence:** `agent/iteration_budget.py:37–45` ([permalink](#)); fallback at `agent/turn_finalizer.py:127–142`. - **Trade-offs:** refunds are an unenforced convention — a forgotten one burns the budget recovery exists to protect. - **Related patterns:** entries 1, 9; circuit breaker.

4. Self-Registering Tool Registry with AST Discovery (plugin registry) - **Status:** CANONICAL. - **Intent:** capabilities as data — schema, handler, toolset, probe — with zero-config discovery. - **Problem/Context:** central import lists rot across five entry points. - **Solution:** modules call `registry.register()` at top level; discovery imports only `tools/*.py` with a module-body `register()` call, behind a text prefilter and `ast.parse`; failures degrade one toolset, never the process. - **Structure:** acyclic chain `registry` — `tools` — `model_tools` (Chapter 4). - **Code evidence:** `tools/registry.py:43–84` ([permalink](#)); trigger at `model_tools.py:194–217`. - **Trade-offs:** import-time side effects every startup; conditional or aliased registration is silently skipped. - **Related patterns:** entries 5, 6; extensibility rings (Chapter 4, Figure 4).

5. Capability Gating at Schema Emission (`check_fn` gating) - **Status:** CANONICAL. - **Intent:** a tool whose probe fails is absent from the schema — physically uncallable. - **Problem/Context:** prompt-level “do not use” is ignorable; runtime capability errors teach workarounds. - **Solution:** `check_fn` evaluated at schema emission behind a 30 s TTL cache with 60 s last-good grace; failures filtered before the OpenAI envelope wrap. - **Structure:** `probe` — `filter` — `envelope` (Chapter 4). - **Code evidence:** `tools/registry.py:143–206, :558–576` ([permalink](#)); probe example `tools/file_tools.py:2104–2107`. - **Trade-offs:** availability up to 30 s stale; a wedged probe hides working tools. - **Related patterns:** entries 4, 18; insight 4.

6. Progressive Disclosure (tool search; three-tier skill loading) - **Status:** CANONICAL. - **Intent:** routing signal stays cheap; capability detail loads on demand. - **Problem/Context:** 74 static tools plus MCP/plugin surfaces and ~120 skills cannot all fit the prompt. - **Solution:** past ~10% of the window, non-core tools collapse behind synthesized `tool_search/tool_describe/tool_call` bridges; skills disclose `index` — `body` — support files, the index riding the stable tier under a mandatory-load preamble. - **Structure:** tier table (Chapter 6). - **Code evidence:** `model_tools.py:551–579` ([permalink](#)); skills index at `agent/prompt_builder.py:1732–1758`. - **Trade-offs:** a badly described capability silently never loads; no embedding retrieval catches the miss. - **Related patterns:** entries 4, 16; Agent Skills standard ^{21 22}.

10.2.2 Context and storage entries (7–11)

7. Three-Tier Prompt Assembly with Prefix-Cache Engineering - **Status:** CANONICAL. - **Intent:** stable bytes first, so provider KV caches hit across turns. - **Problem/Context:** prefix caches match leading bytes; a full miss costs ~75% input-token delta plus latency (Chapter 5). - **Solution:** prompt assembled once per session as stable → context → volatile, rebuilt only after compression; date-only timestamps; Anthropic system_and_3 breakpoints, three rolling with the frontier. - **Structure:** tiers joined `\n\n`; Chapter 5. - **Code evidence:** `agent/system_prompt.py:10–19, :504–511` ([permalink](#)); `agent/prompt_caching.py:109–116`. - **Trade-offs:** information loss (wall-clock needs a tool call); model or credential switches zero the cache. - **Related patterns:** entries 8, 11; context engineering ¹⁶.

8. Byte-Identical History Replay (`api_content` sidecar) - **Status:** CANONICAL. - **Intent:** what turn N sends is exactly what turn N+1 replays. - **Problem/Context:** ephemeral injections must not persist, yet replaying clean content diverges the prefix at the injected point. - **Solution:** composed wire bytes stamped as `api_content`, persisted beside clean content by one helper so drift is impossible; later turns substitute the sidecar. - **Structure:** compose once → persist dual → replay sidecar (Chapter 5). - **Code evidence:** `agent/turn_context.py:50–66` ([permalink](#)); substitution at `agent/conversation_loop.py:1022–1036`. - **Trade-offs:** injected messages stored twice; pure overhead without prefix caching. - **Related patterns:** entries 7, 11; insight 1.

9. Dual-Trigger Compression with Summary Rehydration (compaction) - **Status:** CANONICAL. - **Intent:** bound working-memory growth without losing the task thread. - **Problem/Context:** long sessions overflow; naive truncation severs tool pairs and early goals.¹⁶ - **Solution:** an 85% gateway pass above the in-loop 50% default; prune old results free; keep head plus budgeted tail aligned to tool groups; summarize the middle — re-compression *updates* the prior summary, rehydrated from fossils; swappable engine contract. - **Structure:** Figure 2 restart path (Chapter 3); Chapter 5. - **Code evidence:** `agent/context_compressor.py:4256–4650`; `agent/context_engine.py:89–351` ([permalink](#)). - **Trade-offs:** lossy; each compression invalidates the cached prefix for a turn or two. - **Related patterns:** entries 3, 7; Chapter 5.2.

10. Three-Layer Tool-Result Budgeting (spill-to-store) - **Status:** CANONICAL. - **Intent:** no tool result — and no turn aggregate — may flood the window. - **Problem/Context:** tools return unbounded output; truncation destroys capability. - **Solution:** tools pre-truncate; oversized results spill to sandbox temp files behind ≤1,500-char previews recoverable via `read_file` (pinned infinite, so the recovery tool never spills); a per-turn budget force-spills the rest; budgets scale with the model window. - **Structure:** tool cap → per-result persist → per-turn enforcement (Chapters 5, 9.4). - **Code evidence:** `tools/tool_result_storage.py:3–10` ([permalink](#)); `tools/budget_config.py:84–114`, pin :10–13. - **Trade-offs:** each spill costs a read round-trip; a wrong recovery-tool threshold recreates the spill-read loop. - **Related patterns:** entries 6, 9; research seam (Chapter 9.4).

11. Frozen-Snapshot Context Injection (session-scope freeze) - **Status:** CANONICAL. - **Intent:** inject memory once per session; never mutate the live prompt. - **Problem/Context:** mid-session prompt edits invalidate the prefix cache on every later turn. - **Solution:** `_system_prompt_snapshot` captured at load, never mutated; live writes persist to disk immediately; the snapshot enters the volatile tier; the next session refreshes. - **Structure:**

snapshot — prompt block — session-boundary refresh (Chapter 6). - **Code evidence:** tools/memory_tool.py:123–132, :626–632 ([permalink](#)); injection at agent/system_prompt.py:483–492. - **Trade-offs:** facts learned mid-session are prompt-invisible until next session — freshness traded deliberately. - **Related patterns:** entries 7, 15; insight 1.

10.2.3 Learning-loop and autonomy entries (12–14) plus PIONEER entries

12. Zero-LLM Structured Retrieval over Session History (deterministic FTS recall) - **Status:** CANONICAL. - **Intent:** cross-session recall that is deterministic, inference-free, inspectable. - **Problem/Context:** LLM-summarized recall drifts and costs; embeddings add infrastructure and misses.^{8 9} - **Solution:** session_search runs BM25 over three FTS5 indexes (unicode61; trigram over a tool-role-excluding view — ~90% of bytes; CJK bigram), returning real messages in four shapes as an ordinary tool result. - **Structure:** tool — SQLite — ranked windows (Chapter 6). - **Code evidence:** “No LLM calls anywhere,” tools/session_search_tool.py:1–29 ([permalink](#)); hermes_state.py:1187–1392. - **Trade-offs:** lexical recall misses paraphrases; marker-gated reindexing is deferrable complexity. - **Related patterns:** entries 6, 15; CoALA episodic⁸.

13. Background Reflection Fork with Cache Parity (post-hoc reflection) - **Status:** CANONICAL — Ng’s Reflection¹⁵; the cache-parity fork is the novel mechanic inside this canonical entry. - **Intent:** learn from every turn without competing with it — or doubling its cost. - **Problem/Context:** in-loop critics steal attention; a naive second transcript pass doubles spend.¹ - **Solution:** cadence counters (10 user turns; 10 tool iterations) fire a whitelisted second AI Agent on a daemon thread after successful turns — persistence disabled, compression off, approvals auto-denied — pinning the parent’s cached prompt, session_start, and session_id for cache reuse (~26% saving; single-source in-code figure). - **Structure:** Figure 3 (Chapter 7). - **Code evidence:** agent/background_review.py:617–953; pins :779–789 ([permalink](#)). - **Trade-offs:** without parity, spend roughly doubles on review turns [INFERRED]; the pins couple the fork to prompt internals. - **Related patterns:** entries 14, 16; evaluator-optimizer¹³.

14. Curator Archive-Never-Delete Lifecycle (decay with tombstones) - **Status:** CANONICAL. - **Intent:** decay an accumulating skill library without chores or data loss. - **Problem/Context:** bias-to-act learning produces sediment; deletion makes bad passes unrecoverable. - **Solution:** inactivity-triggered runs (168 h; first run deferred) transition agent-created skills 30 d — stale, 90 d — archive, reuse reactivates; pinned, cron-referenced, never-used, and hub skills exempt; archive only, never delete; five-deep snapshot with rollback. LLM consolidation off by default. - **Structure:** Figure 3 maintenance path (Chapter 7). - **Code evidence:** agent/curator.py:233–283, :305–383 ([permalink](#)); never-delete :17; snapshot :1544–1551. - **Trade-offs:** archives accumulate; the anti-bloat consolidation pass is the one judgment guard left off. - **Related patterns:** entries 13, 16.

15. Agent-Curated File Memory — PIONEER #1 - **Status:** PIONEER. Memory frames converge on vector or provider-backed stores^{8 9}; agent-edited bounded Markdown without a similarity index has no clean precedent. - **Intent:** durable cross-session semantic memory the agent curates, at zero infrastructure and cache cost. - **Problem/Context:** vector memory adds weight and misses; append-to-prompt memory busts caches. - **Solution:** MEMORY.md (2,200 chars) / USER.md (1,375), \$-delimited; one bounded memory tool edits by unique substring

under locks, atomic renames, drift guards — deliberately no read action. - **Structure:** tool → atomic file → frozen snapshot (entry 11) → next session (Chapter 6). - **Code evidence:** tools/memory_tool.py:3–25, :55–57 ([permalink](#)); substring errors :398–399. - **Trade-offs:** no semantic recall (entry 12 covers recall); the ceiling holds only what the model judged worth compressing; quality is model-dependent. - **Related patterns:** entries 11, 12; OpenClaw workspace files as nearest peer^{2 3}.

16. Self-Authored SKILL.md Procedural Memory — PIONEER #2 - **Status:** PIONEER. The skills standard assumes human authors,^{21 22} and ReAct’s named limitation is unpersisted learning¹; self-authorship under provenance and decay is unframed. - **Intent:** convert episodic success into reusable, portable procedural knowledge. - **Problem/Context:** trajectories end; each session rediscovers procedures. - **Solution:** the reflection fork and /learn author standard SKILL.md packages (≤60-char description, fixed sections, literal author: Hermes); ContextVar provenance marks agent_created; read-before-write; 100 KB cap; decay applies only to agent sediment, never user or hub skills. - **Structure:** Figure 3 (Chapter 7); Figure 4 ring position (Chapter 4). - **Code evidence:** tools/skill_provenance.py:37–45; tools/skill_manager_tool.py:488, :1421–1427 ([permalink](#)). - **Trade-offs:** wrong-but-plausible procedures accumulate unmeasured; one-offs harden into self-cited refusals. - **Related patterns:** entries 6, 13, 14.

17. Natural-Language Cron with Isolated Sessions — PIONEER #3 - **Status:** PIONEER. Proactivity frames offer heartbeats and fixed cron^{20 7}; the model as schedule parser plus a fresh capability-stripped session per fire is unframed. - **Intent:** proactivity as a tool the agent itself wields, safely. - **Problem/Context:** fixed cron is rigid; heartbeats block and drift; unattended agents can recursively self-schedule. - **Solution:** the model emits one of four validated schedule shapes; each fire builds a fresh historyless AIAgent with skills injected; next_run_at advances before execution (at-most-once); cron/messaging/clarify toolsets stripped in code; provider pinned fail-closed. - **Structure:** fresh session → delivery → next-run update (Chapter 8). - **Code evidence:** cron/jobs.py:512–609 ([permalink](#)); cron/scheduler.py:156–176, :4029–4035. - **Trade-offs:** well-formed-but-wrong schedules fail silently; a post-advance crash is a missed run, never a duplicate. - **Related patterns:** entries 13, 18; heartbeat/cron frames^{20 7}.

18. Capability-Subtraction Guardrails — PIONEER #4 - **Status:** PIONEER. HITL frames center approval gates^{18 19}; schema-level capability removal as the *primary* control — instruction only as backup — is argued as distinct; the strongest transfer case. - **Intent:** make runaway or recursive behavior physically impossible, not discouraged. - **Problem/Context:** prompt-level refusals are ignorable under injection; spawned contexts inherit the parent’s full surface. - **Solution:** DELEGATE_BLOCKED_TOOLS strips delegation, memory, cron, messaging from subagents; cron sessions lose three toolsets in code with check_fn as a second layer; yolo/redact toggles freeze at import. - **Structure:** Chapter 8’s three-surface table; Chapter 9.1. - **Code evidence:** tools/delegate_tool.py:47–54 ([permalink](#)); cron/scheduler.py:156–176; tools/approval.py:32–35. - **Trade-offs:** subtraction is coarse — it removes legitimate uses too — and each new spawn context must re-derive its strip. - **Related patterns:** entries 5, 13, 17; insight 4.

10.3 Coverage Map and Deliberate Absences

10.3.1 Coverage-map walk-through

Figure 5 is the chapter’s anchor artifact: every entry points at the module(s) instantiating it, and every core module points back at its pattern(s). The table is the same map in tabular form.

Canonical pattern	Coverage	Hermes implementation (source-verified)
ReAct loop (reason+act)	FULL	agent/conversation_loop.py — sync state-machine loop, iterationBudget 90/50, tool-less summary at exhaustion
Orchestrator-workers	FULL	tools/delegate_task → fresh AIAgent per child, depth-1 default, summary fan-in, daemon-pool ≤3 concurrent
Parallelization (voting)	FULL	agent/moa_loop.py — Mixture-of-Agents fan-out + aggregator synthesis per turn
Routing	FULL	provider fallback chain + credential rotation (error_classifier ~25 FailoverReasons); gateway channel routing
Reflection	FULL	background reflection fork (memory nudge /10 turns, skill review /10 tool iters) — background_review.py
Memory — semantic	FULL	MEMORY.md / USER.md frozen snapshots, memory tool \$-entries (CoALA semantic)
Memory — episodic	FULL	SQLite state.db + FTSS session_search (zero-LLM recall, 4 query shapes)
Memory — procedural	FULL	SKILL.md library + progressive disclosure (<available_skills> index → skill_view on demand)
Memory — working	FULL	tiered context: byte-stable system prompt + history + ephemeral call-time injection
Human-in-the-loop	FULL	fail-closed approval CoR (tools/approval.py) — once/session/always/deny; yolo frozen at import
Planning	PARTIAL	no explicit planner; planning emerges from skills + todo tool; MoA advisory
Evaluator-optimizer	PARTIAL	verify-on-stop evidence ledger (verification_stop.py); smart-approval aux LLM
Prompt chaining	WEAK	no native chain construct; approximated via skills + cron pipelines
Tool use	FULL	74 native tools, registry + AST discovery, MCP merge, 3-layer result budgeting
Proactivity / scheduling	FULL+PIONEER	NL cron → isolated fresh-agent sessions, toolset-stripped (cron/scheduler.py:156-176)
Self-improvement	PIONEER	agent-authored SKILL.md via reflection fork + Curator gardening + external GEPA (PR-gated)
Agent-curated file memory	PIONEER	two markdown files + nudge cadence — zero-infra persistent memory
Profile isolation	PIONEER	per-profile HERMES_HOME (config, memory, sessions, PID) — agent-level multitancy
DAG workflow graphs	ABSENT (by design)	loop-centric: complexity pushed into skills/memory/cron instead of orchestration topology
Planner-executor split	ABSENT (by design)	single agent plans+acts; orchestrator role opt-in only

Taxonomy sources: Anthropic “Building effective agents” (5 workflow patterns + agent split) · Ng’s 4 agentic patterns · CoALA memory taxonomy.
PIONEER = pattern absent/weak in canonical lists, defined in this study as a handbook entry (Intent/Problem/Solution/Structure/Trade-offs).

Figure 5: Patterns-to-modules coverage map — each catalog entry points at its primary module(s); core modules point back at their pattern(s); substrate modules (environments, providers, approval) sit beneath the patterns they carry.

Pattern (entry)	Primary module(s)	Host chapter
Loop-as-State-Machine (1)	agent/conversation_loop.py, agent/turn_retry_state.py	Chapter 3
Prologue/Epilogue Turn Seams (2)	agent/turn_context.py, agent/turn_finalizer.py	Chapter 3
Iteration Budget Consume/Refund (3)	agent/iteration_budget.py	Chapter 3
Self-Registering Registry (4)	tools/registry.py, model_tools.py	Chapter 4
Capability Gating, check_fn	tools/registry.py:143–206,	Chapter 4

Pattern (entry)	Primary module(s)	Host chapter
(5)	558–576	
Progressive Disclosure (6)	model_tools.py, agent/prompt_builder.py	Chapters 4, 6
Three-Tier Prompt Assembly (7)	agent/system_prompt.py, agent/prompt_caching.py	Chapter 5
Byte-Identical Replay (8)	agent/turn_context.py, agent/conversation_loop.py	Chapter 5
Dual-Trigger Compression (9)	agent/ context_compressor.py, agent/context_engine.py	Chapter 5
Three-Layer Result Budget (10)	tools/tool_result_storage.py, tools/budget_config.py	Chapters 5, 9.4
Frozen-Snapshot Injection (11)	tools/memory_tool.py	Chapters 5, 6
Zero-LLM Retrieval (12)	hermes_state.py, tools/session_search_tool.py	Chapter 6
Background Reflection Fork (13)	agent/background_review.py	Chapter 7
Curator Lifecycle (14)	agent/curator.py, agent/curator_backup.py	Chapter 7
Agent-Curated File Memory (15)	tools/memory_tool.py	Chapter 6
Self-Authored SKILL.md (16)	tools/skill_manager_tool.py, tools/skill_provenance.py	Chapters 6, 7
NL Cron, Isolated Sessions (17)	cron/jobs.py, cron/scheduler.py	Chapter 8
Capability-Subtraction (18)	tools/delegate_tool.py, cron/scheduler.py, tools/approval.py	Chapters 8, 9

Reading the map in both directions is the point. Pattern-to-module, no entry is homeless: every mechanism resolves to at least one file at the snapshot, which makes the admission rule of §10.1.1 checkable. Module-to-pattern, the dense nodes match where the chapters put their weight: `conversation_loop.py` carries entries 1, 2, 8; `tools/registry.py` carries 4 and 5; `tools/memory_tool.py` carries 11 and 15; `cron/scheduler.py` carries 17 and part of 18. Two honest gaps in the reverse direction deserve naming. First, the platform substrate — `tools/environments/`, the provider layers, the approval chain — has no pattern entry: it is the replace surface the patterns run on, treated in Chapter 9 as seam rather than mechanism, and reappears in Part B as the core’s expectations. Second, the verify-on-stop ledger (`agent/verification_evidence.py`, `agent/verification_stop.py`) sits inside entry 13’s host subsystem but is deliberately not an entry: it verifies truthfulness within a turn, not a reusable cross-turn mechanism — Chapter 7.3.1 scopes it as a floor, not a fitness function.

10.3.2 Deliberately absent patterns

The following canonical patterns are not instantiated at the snapshot. Absence is information: each states what the absence means for the clone decision, in the same neutral register as the catalog. The set is consistent with the loop-centric thesis — complexity pushed into data the loop reads, not orchestration scaffolding — not a list of gaps.

- **Planner-executor split.** Planning is loop-implicit, externalized only through kanban/goals tools. Clone decision: skip the planner; the stale-plan failure mode arrives only if you add one.
- **DAG/workflow-graph orchestration.** Control flow lives in the model; skills encode procedures as prose.^{13 1} Clone decision: do not buy a graph framework; loop plus registry replaces it.
- **Tree search / backtracking** (ToT, LATS). Traded for latency and simplicity; tools/checkpoint_manager.py gives environment-level undo. Clone decision: copy checkpoint/rollback if your environment is destructive; skip search.
- **Multi-agent debate / role-play.** MoA covers many-perspectives on demand; persistent swarms would conflict with the profile-isolated single-identity model [INFERRED]. Clone decision: defer MoA; add only if voting measurably helps your task mix.
- **In-loop evaluator-optimizer pair.** Replaced by the reflection fork and verify-on-stop; fitness-measured self-modification lives only in the external, PR-gated self-evolution repo.^{10 11} Clone decision: keep it that way — in-loop fitness would demand the human-review gates the project reserves for the companion.
- **Vector/embedding memory as default.** File memory plus FTS5 sessions cover recall; vector stores are opt-in plugins. Clone decision: start dependency-free; adopt embeddings only when recall misses are observed.
- **Exactly-once cron delivery.** At-most-once by design: advance-before-execute makes a crash a missed run, never a duplicate (cron/scheduler.py:4029-4035). Clone decision: accept auditable misses; exactly-once costs a durable intent log plus idempotent side effects at every target.

10.4 Part B — Clone Roadmap

10.4.1 Scope and seam

The roadmap clones the **agent core only**; channels, the gateway, ACP, batch, and the API server are named solely to be excluded — entry points, not core (Chapters 1, 9). The core *expects*: turn input (a user message plus optional system context); an environment honoring the two-method BaseEnvironment contract (`_run_bash`, `cleanup`; Chapter 9.2); approval decisions via a callback the gate invokes fail-closed; and a delivery target for output and async completions. The core *exposes*: lifecycle events (the AIAgent callback surface, Chapter 3), on-disk artifacts (memory files, skills, trajectories), and `state.db` (sessions, messages, lineage, cost; Chapter 6). A host supplying the four expectations and consuming the three exposures can swap entry points without touching the core — how one AIAgent serves five front ends.

10.4.2 Minimal viable core

The smallest subset that still runs one full turn: provider path, loop, registry with a handful of tools, one environment, approval — steps 1–4 below, step 5 strongly advised. Everything later is additive data the loop reads. This is insight 7’s conclusion, reached independently by every dimension’s clone guidance: the minimal viable Hermes is roughly 10–15% of the codebase, converging on the same dependency order. Chapter 3 anchors the claim: the loop’s load-bearing regions are small and extractable; the rest of its ~3,900 lines is additive robustness with named, deferrable failures.

10.4.3 Staged 10-step build order

Figure 6 shows the order as a dependency graph. Each step has a done-state and one primary risk. LOC figures are convergent estimates from the per-dimension clone-guidance sections (insight 7) — estimates, never measurements. Where per-chapter clone notes assume broader scope they run higher (Chapter 4: ~600 registry plus ~1,500 tools; Chapter 6: ~300 skills; Chapter 7: ~400 fork); the figures below assume the minimal core of §10.4.2.

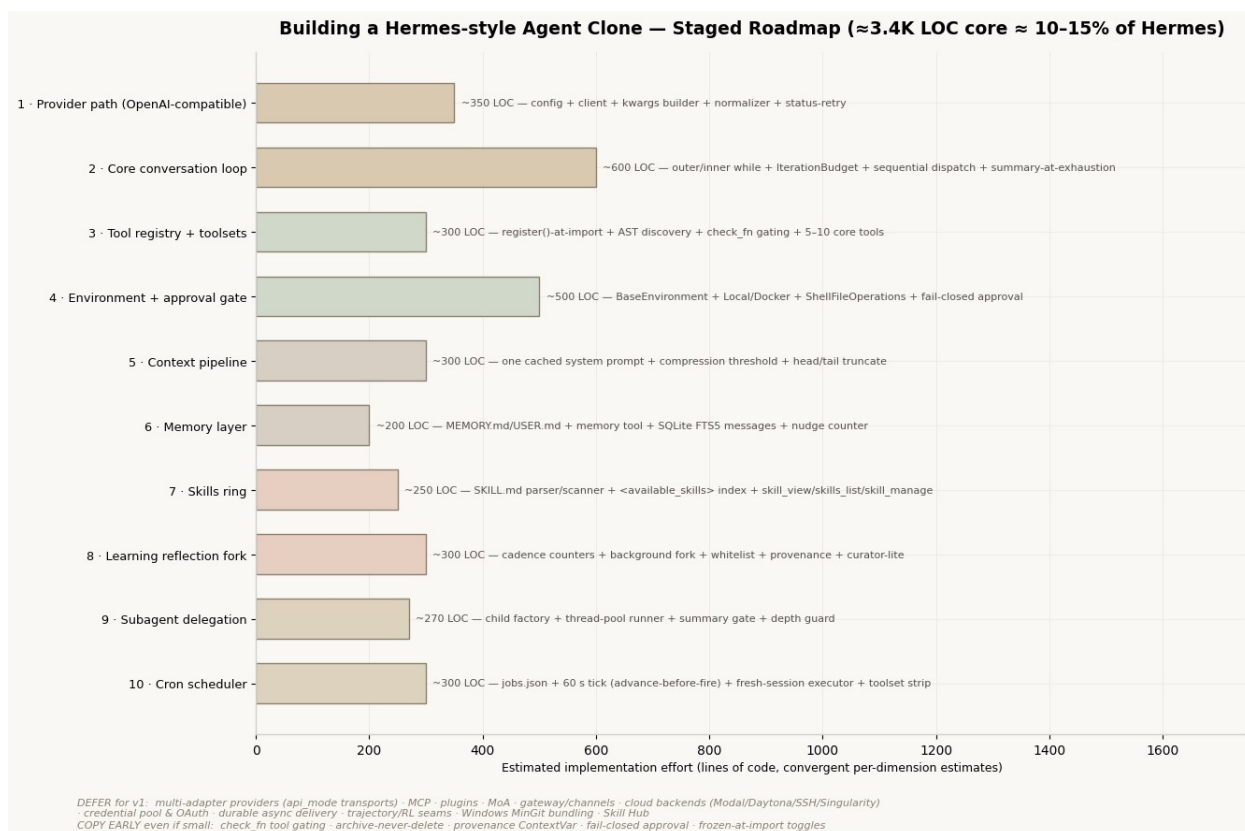


Figure 6: Clone roadmap — staged build order with dependencies; steps 1–4 form the minimal viable core, steps 5–7 add the data layers, steps 8–10 add learning and autonomy.

1. **Provider path (~350).** One streaming chat_completions transport with basic retry; skip auth registry and plugins. Done: one streaming call round-trips a tool schema. Risk: streaming/tool-call edge cases. Keep the OpenAI-shaped representation as the day-one invariant (Chapter 9).

2. **Core loop (~600)**. Outer budget while, tool-call round-trips, IterationBudget with summary-call fallback, sequential dispatch. Done: one turn runs end to end. Risk: consume/refund correctness (Chapter 3).
3. **Registry + a handful of tools (~300)**. Import-time registration, OpenAI-format schemas, JSON-string handler contract; a static import list is an acceptable v1 substitute for the AST gate. Done: the model calls a file tool in the loop. Risk: handler-contract drift — errors as {"error": ...}, never raised (Chapter 4).
4. **Environment + approval (~500)**. One local BaseEnvironment honoring the two-method contract; fail-closed approval with hardline floor and frozen-at-import toggles. Done: a dangerous command is gated; a timeout denies. Risk: containment trust concentrated in one backend — state it, as Chapter 9 does.
5. **Context pipeline (~300)**. Three-tier assembly plus dual-trigger compression behind a swappable engine contract. Done: a long session compresses without losing the task. Risk: boundary bugs splitting tool pairs (Chapter 5).
6. **Memory layer (~200)**. MEMORY.md/USER.md, \$-delimited, char budgets, substring targeting, atomic writes, frozen-snapshot injection. Done: the agent edits its own MEMORY.md. Risk: none structural; staleness is by design (Chapter 6).
7. ****Skills** (₂₅₀). 150-line front-matter parser, one discovery root, skills_list/skill_view/skill_manage, index into the stable tier. Done: skill_view loads a body on demand. Risk: ignored load preamble collapses routing (Chapter 6).
8. **Learning fork (~300)**. Cadence counters, whitelisted second agent with persistence disabled, curator-lite stale/archive with snapshot/rollback. Done: a fork writes a skill after ten iterations. Risk: fork lifecycle bugs; if time-pressed, start with /learn-style foreground distillation (Chapter 7).
9. **Delegation (~270)**. Child factory (fresh agent, goal+context prompt, parent toolsets minus blocklist), pool runner, summaries-only fan-in, depth-1 guard. Done: parent receives bounded summaries, never child transcripts. Risk: runaway recursion if the ~50-line blocklist is omitted — implement it day one (Chapter 8).
10. **Cron (~300)**. JSON job store with atomic writes, tick loop, fresh-session executor with toolset strip, advance-before-execute. Done: first cron fire runs an isolated session. Risk: silent schedule mistranslation — bound it with the four-shape whitelist (Chapter 8).

10.4.4 *Defer-list and copy-early guards*

Defer, with justification: **MCP and Tool Search** — an external-process trust surface adding failure modes before capability. **Provider plugins beyond one** — one transport proves the invariant; the 33-plugin machinery and OAuth chain come later. **Skills Hub** — untrusted-text intake is a security project of its own. **execute_code RPC scripting** — a second dispatch surface presupposing a stable dispatcher. **Three-index FTS5** — solves scale the clone does not yet have. **MoA, gateway/ACP bindings, batch_runner, datagen** — product and research surfaces outside the core by definition. **GEPA/DSPy evolution** — external to the original repo by design; replicating it is a research program, not a clone step.^{10 11}

Copy early — each is cheap and prevents an entire bug class: check_fn gating at schema emission (~50 lines; Chapter 4); archive-never-delete with snapshot/rollback (Chapter 7); the provenance flag on agent-authored writes — unretrofitable once autonomous writes exist

(Chapter 6); fail-closed approval with the hardline floor below yolo (Chapter 9); frozen-at-import toggles, so no in-process action can lower a safety setting (Chapter 9).

10.4.5 Decision register

Preserve verbatim — the interface is the value: the tool schema shape and handler contract; the `SKILL.md` format, because interop is the bet (Chapter 6); the `api_content` sidecar discipline, *if* the target provider offers prefix caching; the OpenAI-shaped message representation (Chapter 9); the consume/refund budget discipline. **Replace freely** — the interface is incidental: state storage (SQLite/FTS5 is a choice behind the session API, not an invariant); delivery routing; the plugin system; profile multiplexing. The insight-1 fork prices this: targeting a provider without prefix caching deletes roughly 15–20% of the system’s complexity — sidecars, replay machinery, frozen snapshots, cache-parity fork, breakpoint placement all collapse into “send the history” (Chapter 5).

Re-verify before building, given drift from the July 2026 snapshot: self-improvement mechanics and provider integrations are the most volatile surfaces; the static tool count (74 at snapshot, Medium-confidence single counter, Chapter 4) versus the README’s “40+”; documentation locating the loop in `run_agent.py`, stale since the god-file decomposition (Chapter 3, conflict zone C1); the surviving research seams after the RL amputation (Chapter 9.4). Pin your own snapshot and re-run the count scripts rather than trusting any article — this one included.

10.4.6 Phasing table

Phase	Milestone (done-state)	Copy / stub / re- implement	Effort (est.)	Risk
1 — Core turn (steps 1–2)	One full turn runs end to end	Copy transport shape and IterationBudget; re-implement minimal loop	~950 LOC	Streaming edge cases; refund correctness
2 — Capability (steps 3–4)	Model calls gated tools in a real environment	Copy registry contract and approval floor; stub AST gate with a static list	~800 LOC	Handler- contract drift; containment in one backend
3 — Data layers (steps 5–7)	Session compresses; agent edits memory; skills load on demand	Copy tier assembly, memory tool, front-matter parser	~750 LOC	Compression boundary bugs; ignored load preamble
4 — Learning & autonomy (steps 8–10)	Fork writes a skill; subagent fan-in; first cron fire	Re-implement fork with guardrails verbatim; copy blocklist; port	~870 LOC	Fork lifecycle; recursion without blocklist; schedule

Phase	Milestone (done-state)	Copy / stub / re- implement	Effort (est.)	Risk
		parse_schedule		mistranslation

The table compresses §10.4.3 into buildable phases, and the compression is where its numbers should be read with care. Effort figures are convergent estimates, not measurements; per-chapter clone notes run higher where they assume fuller scope (Chapter 4's ~600 + ~1,500; Chapter 6's ~300 skills; Chapter 7's ~400), so treat the totals as a floor with roughly 1.5× headroom, not a budget. The copy/stub/re-implement column carries the roadmap's real opinion: copy wherever a contract is load-bearing (budget, approval floor, blocklist, provenance), stub wherever machinery outruns need (AST discovery, FTS indexes, async delivery), and re-implement only where the original's size is incident-driven rather than design-driven — the loop and the fork, whose thousands of lines encode provider incidents a fresh codebase has not yet had. Phases 1–2 are the point of no return: later mistakes are additive and recoverable, while a mistake in the refund discipline or the approval floor is silent and systemic. The natural stopping point for most clones is the end of Phase 3; Phase 4 is where the system starts writing its own future prompts, and its guards are the non-negotiable part.

The forward-looking implication closes the study. Individual entries are already diffusing — SKILL.md is a cross-vendor standard, and subagent memory scopes show convergent evolution elsewhere^{21 5} — but the durable transfer is not any single pattern; it is the governing bet that produced them: one boring loop, behavior encoded as auditable data, autonomy governed by subtraction. A clone built on this roadmap inherits an architecture whose improvement is measured by an external, human-gated pipeline rather than by in-repo self-modification — and the re-verify list of §10.4.5 is where to watch that bet evolve.

References

- [1] SCHLUNTZ E, ZHANG B. Building effective agents[EB/OL]. (2024-12). <https://www.anthropic.com/research/building-effective-agents>
- [2] A Two-Dimensional Framework for AI Agent Design Patterns: Cognitive Function × Execution Topology[EB/OL]. (2026-05-24). <https://arxiv.org/html/2605.13850v2>
- [3] Agentic Design Patterns: A System-Theoretic Framework[EB/OL]. (2026-01-27). <https://arxiv.org/html/2601.19752v1>
- [4] Octamem. The 4 Types of AI Agent Memory: Semantic, Episodic, Procedural & Working[EB/OL]. (2026-07-01). <https://octamem.com/blog/types-of-ai-agent-memory>
- [5] mer.vin. AI Agent Memory Explained: Episodic, Semantic, Procedural and Working Context[EB/OL]. (2026-05-20). <https://mer.vin/2026/05/ai-agent-memory-explained-episodic-semantic-procedural-and-working-context/>
- [6] Anthropic. Effective context engineering for AI agents[EB/OL]. (2025-09-29). <https://github.com/jcosta33/corpus-skills/blob/main/docs/sources.md>

- [7] YAO S, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. (2022-10-06). <https://arxiv.org/pdf/2210.03629>
- [8] KINNEY S. Agent loops[EB/OL]. (2026-03-19). <https://github.com/stevekinney/stevekinney.net/blob/main/writing/agent-loops.md>
- [9] CreateOS. Human-in-the-Loop AI Agents: When Approval Gates Matter[EB/OL]. (2026-06-28). <https://createos.sh/blogs/human-in-the-loop-ai-agents>
- [10] BetterClaw. AI Agent Guardrails: Human Approval Without Killing Speed[EB/OL]. (2026-06-11). <https://www.betterclaw.io/blog/ai-agent-human-approval-guardrails>
- [11] MindStudio. What Is the Agentic OS Heartbeat Pattern?[EB/OL]. (2026-04-04). <https://www.mindstudio.ai/blog/agentic-os-heartbeat-pattern-proactive-ai-agent>
- [12] Clawlify. Heartbeat vs Cron in OpenClaw[EB/OL]. (2026-07-09). <https://www.clawlify.com/resources/heartbeat-vs-cron-openclaw-guide-2026>
- [13] Agent Skills open standard[EB/OL]. (2026). <https://agentskills.io/home>
- [14] Harness Engineering for Agentic AI Coding Tools: An Exploratory Study[EB/OL]. (2026-03-28). <https://arxiv.org/html/2602.14690v5>
- [15] Nous Research. hermes-agent-self-evolution: README + PLAN.md[EB/OL]. (2026-03-09). <https://github.com/NousResearch/hermes-agent-self-evolution>
- [16] The Agent Report. Hermes Agent Self-Evolution: Nous Research Ships Genetic Prompt Optimization with DSPy + GEPA[EB/OL]. (2026-06-08). <https://the-agent-report.com/2026/06/hermes-agent-self-evolution-dspy-gepa-june2026/>
- [17] Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems[EB/OL]. (2026-07-02). <https://arxiv.org/html/2604.14228v2>
- [18] Modern AI Guide. Programmatic prompt optimization (DSPy, GEPA)[EB/OL]. (2026). <https://modernaiguide.dev/docs/stack/prompt-optimization>
- [19] freeCodeCamp. How to Build and Secure a Personal AI Agent with OpenClaw[EB/OL]. (2026-04-06). <https://www.freecodecamp.org/news/how-to-build-and-secure-a-personal-ai-agent-with-openclaw/>
- [21] CHEN G. Claude Code's Five-Layer Architecture[EB/OL]. (2026-05-02). <https://chenguangliang.com/en/posts/claude-code-five-layer-architecture/>
- [22] Subagent persistent memory scopes (user/project/local); MoA/voting parallelization per Anthropic taxonomy[EB/OL]. (2026-05-02). <https://chenguangliang.com/en/posts/claude-code-five-layer-architecture/>
- [23] vectara. AutoGPT planning failures case study (awesome-agent-failures)[EB/OL]. (2025-08-20). <https://github.com/vectara/awesome-agent-failures/blob/main/docs/case-studies/autogpt-planning-failures.md>

[24] Nous Research. hermes-agent: a self-improving AI agent (source code, commit 4c9628e) [EB/OL]. 2026. <https://github.com/NousResearch/hermes-agent>